

BATCH FRAMEWORKS

YARN

Hadoop's MapReduce

Apache Spark

YARN

Yet Another Resource Negotiator

YARN

We talked about Map and Reduce tasks, but

- Where do I run them?
- What if some tasks fail?
- What if I want to run multiple jobs?

The Resource Negotiator

- Has a global view of the resources available in the cluster
- Decides *where* and *when* each unit of work should be allocated and with how many resources (CPU & RAM)
- Uses a scheduling policy to manage concurrency
- Avoids over-instantiation of processes on the same worker
- Handles fault-tolerance

YARN

Apache YARN is Hadoop's cluster resource management

- Introduced in Hadoop 2 to improve the MapReduce implementation
- It is **general enough to support other distributed computing paradigms**

YARN provides **APIs** for requesting and working with cluster resources

- These APIs are typically **used by distributed computing frameworks** (e.g., MapReduce, Spark), NOT by user code

YARN: main daemons

YARN provides its core services via two types of long-running daemons:

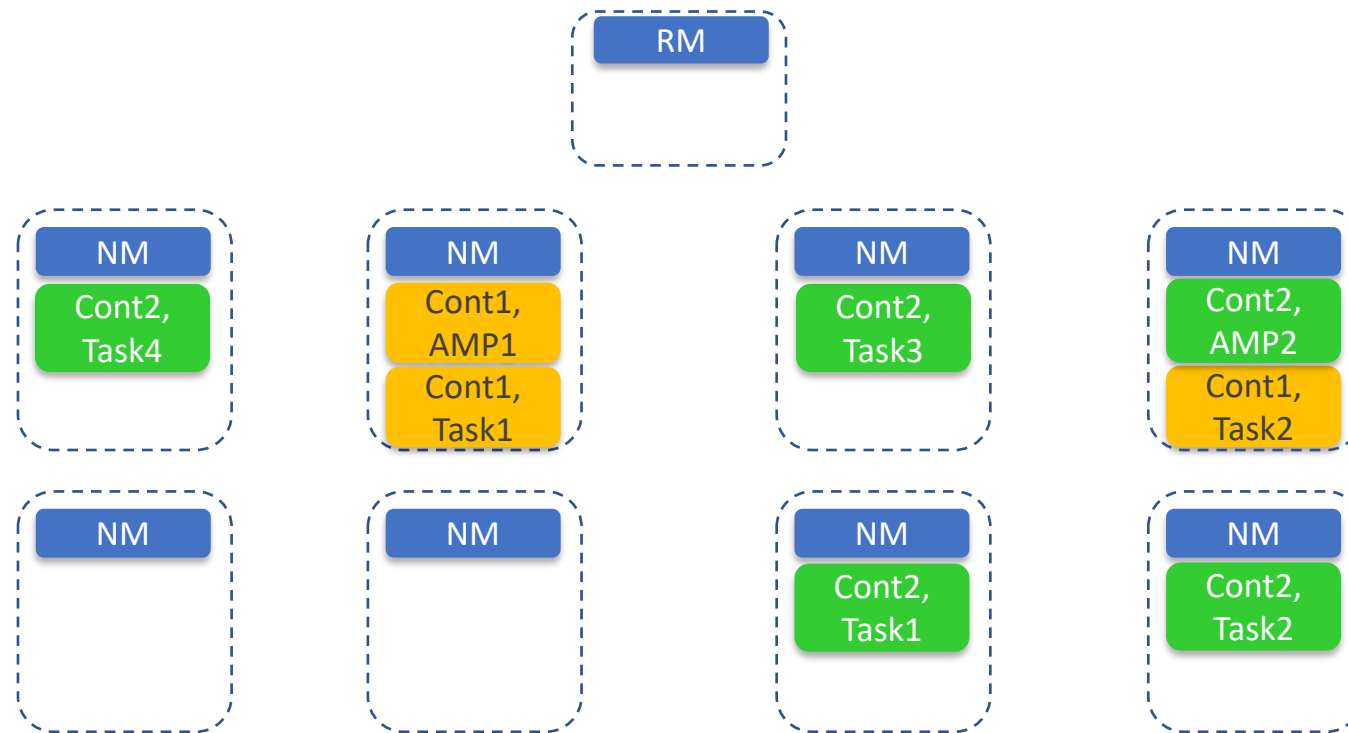
- A global (one per cluster) **Resource Manager** (RM)
 - It is the ultimate authority that arbitrates resources among all the applications
- A per-node slave, the **Node Manager** (NM)
 - It is responsible for *containers*
 - A **container** is an abstract entity, used to execute an application-specific process with a constrained set of resources
 - The NM monitors resource usage of containers (e.g., CPU, memory) and reports them to the RM

YARN: main daemons

Upon request from a client, the RM finds a NM that can launch the *application master process* (AMP) in a container

- The AMP is a special process that controls the execution of an application
- It is in charge of requesting further containers to actually run the distributed computation
 - Allocation/de-allocation requests can take place in a dynamic fashion
- Offloads the burden from the RM

YARN: main daemons



Data locality

Exploit cluster topology and data block replication to apply the data locality principle

*When computations involves large set of data,
it is cheaper (i.e. faster) to **move code to data**
rather than data to code*

The following cases respect the order the resource manager prefers:

1. Process and data on the same node
2. Process and data on the different node of the same rack
3. Process and data on different racks of the same data center
4. Process and data on different racks of the different data centers

YARN: Walkthrough

Application execution consists of the following steps:

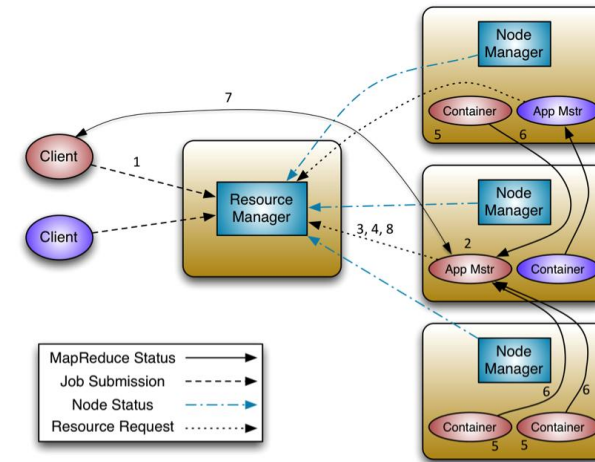
- Client submits application
- RM bootstraps the AMP instance
- AMP instance manages the application execution

1. A client program **submits the application**, including the necessary specifications to launch the application-specific AMP itself

2. The RM assumes the responsibility to **negotiate** a specified container in which to start the AMP and then **launches** it

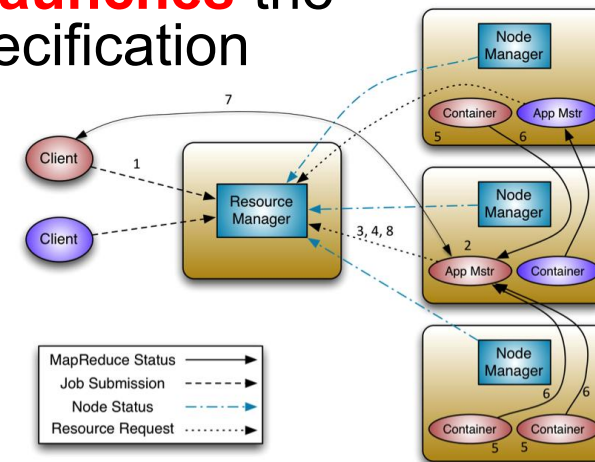
3. The AMP, on boot-up, **registers with the RM**

- After asking details to RM, the client program is able to directly communicate with its own AMP



YARN: Walkthrough

4. During normal operation, the AMP **negotiates** appropriate resource containers
5. On successful container allocations, the AMP **launches** the container by providing the container launch specification to the NM (including information to enable communications with the AMP)
6. The application code executing within the container **provides necessary information** (progress, status etc.) to its AMP
7. During the application execution, the client that submitted the program **communicates** directly with the AMP to get status, progress updates, etc.
8. Once the application is complete, the AMP **deregisters** with the RM and shuts down, allowing its own container to be repurposed



YARN: Scheduler

YARN provides a choice of schedulers and configurable policies

FIFO Scheduler

- Simple to understand, no configuration needed
- Not suitable for shared clusters

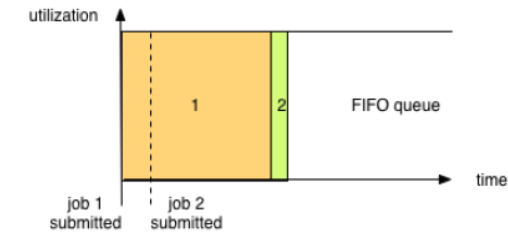
Capacity Scheduler

- Reserves a fixed amount of capacity to each job

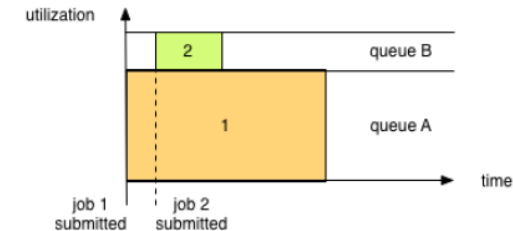
Fair Scheduler

- Dynamically balances the available resources between all running jobs

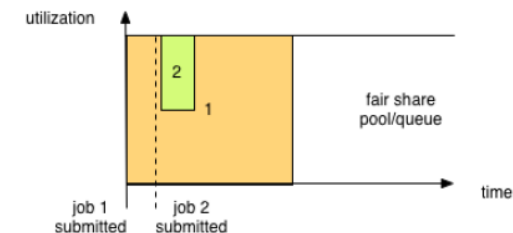
i. FIFO Scheduler



ii. Capacity Scheduler



iii. Fair Scheduler



Hadoop's MapReduce

The implementation of MapReduce in the Hadoop framework

Disambiguation of MapReduce

"MapReduce is a programming model and an associated implementation for processing and generating large data sets.

Users specify a map function that processes a key/value pair to generate a set of intermediate key/value pairs, and a reduce function that merges all intermediate values associated with the same intermediate key."

-- Dean J., Ghemawat S. (Google)

Hadoop MapReduce is an open-source implementation of the MapReduce programming model

MapReduce program

A MapReduce program, referred to as a **job**, consists of:

- Code for Map and Reduce
- Configuration parameters (input/output directories on the underlying distributed file system)

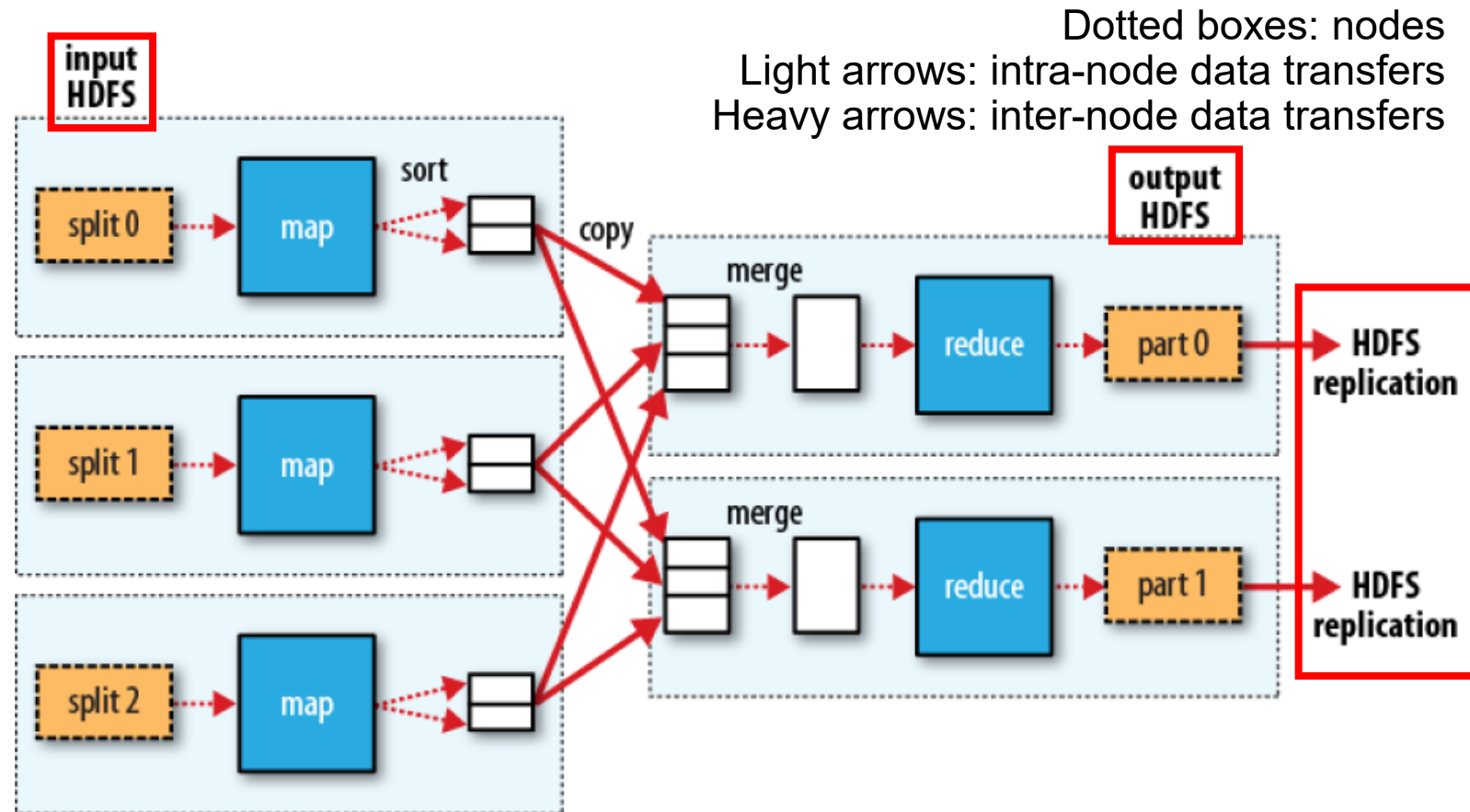
Each MapReduce job is divided by the system into smaller units called **tasks**

- Map tasks
- Reduce tasks

The tasks are scheduled using YARN and run on nodes in the cluster

- If a task fails, it will be automatically rescheduled to run on a different node

MapReduce process



A Map in Java

```
public class WordCountMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    private static final IntWritable one = new IntWritable(1);  
    private Text word = new Text();  
  
    public void map(LongWritable key, Text value, Context context)  
        throws IOException, InterruptedException {  
        String line = value.toString();  
        StringTokenizer tokenizer = new StringTokenizer(line);  
  
        while (tokenizer.hasMoreTokens()) {  
            word.set(tokenizer.nextToken());  
            context.write(word, one);  
        }  
    }  
}
```

A Reduce in Java

```
public class WordCountReducer extends Reducer<Text, IntWritable, Text, IntWritable> {  
    public void reduce(Text key, Iterable<IntWritable> values, Context context)  
        throws IOException, InterruptedException {  
        int sum = 0;  
        for (IntWritable value : values) {  
            sum += value.get();  
        }  
        context.write(key, new IntWritable(sum));  
    }  
}
```

The Job in Java

```
public class WordCount {  
    public static void main(String[] args) throws Exception {  
        Job job = new Job(new Configuration(), "WordCount");  
        job.setJarByClass(WordCount.class);  
        job.setMapperClass(WordCountMapper.class);  
        job.setReducerClass(WordCountReducer.class);  
        FileInputFormat.addInputPath(job, new Path(args[0]));  
        FileOutputFormat.setOutputPath(job, new Path(args[1]));  
        job.setOutputKeyClass(Text.class);  
        job.setOutputValueClass(IntWritable.class);  
        job.waitForCompletion(true);  
    }  
}
```

MapReduce execution

Important idea behind MapReduce is separating *the what* of distributed processing from *the how*

- We have seen the former, let's see the latter

The developer launches the job on the client's JVM, which contacts the YARN RM to submit the application

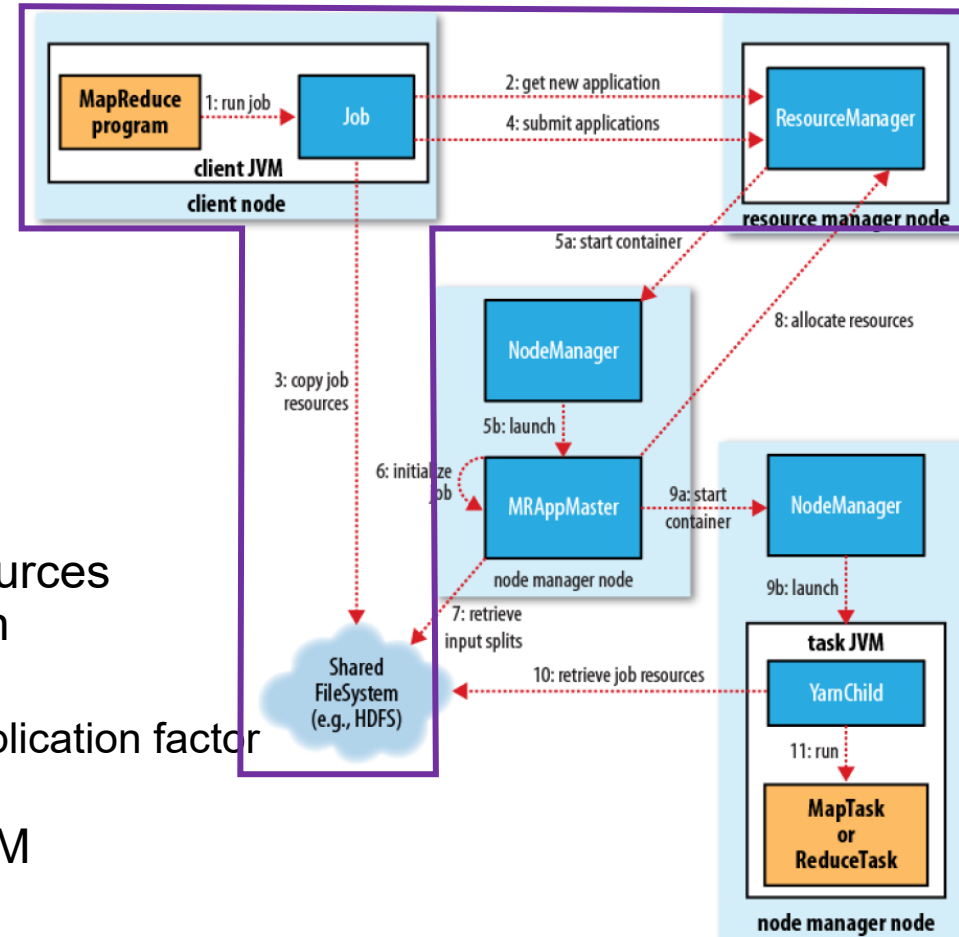
The execution framework takes care of everything else...

- It transparently handles all aspects of distributed code execution
- On clusters ranging from a single node to a few thousand nodes

MapReduce execution

Job Submission

- 1: The MapReduce program calls either the submit() or waitForCompletion() method to create the Job
- 2: The Job contacts the RM to get an application ID
- 3: The Jobs copies to the DFS the resources needed to run the job (i.e., configuration file, JAR file, input splits metadata)
 - The job JAR is copied with a default replication factor of 10, so that NMs can easily access it
- 4: The application is submitted to the RM



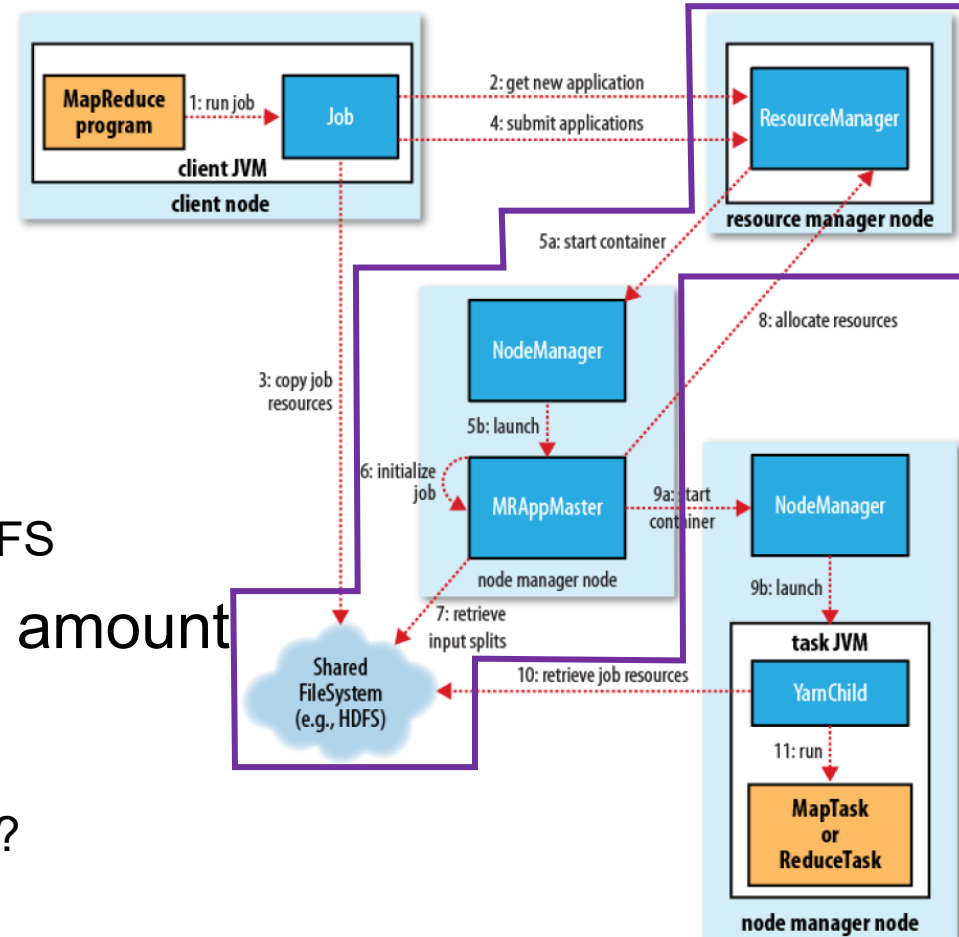
MapReduce execution

Job Initialization

- 5a: The YARN Scheduler (called by the RM) allocates a container on a NM to host the AMP
 - AMP = MRAppMaster in the picture
- 6: The AMP is launched in the container
- 7: Input split data are identified on the DFS

At this point, the AMP evaluates the amount of resources required to run the job

- How many containers are necessary?
- Is the parallelization worth the overhead?



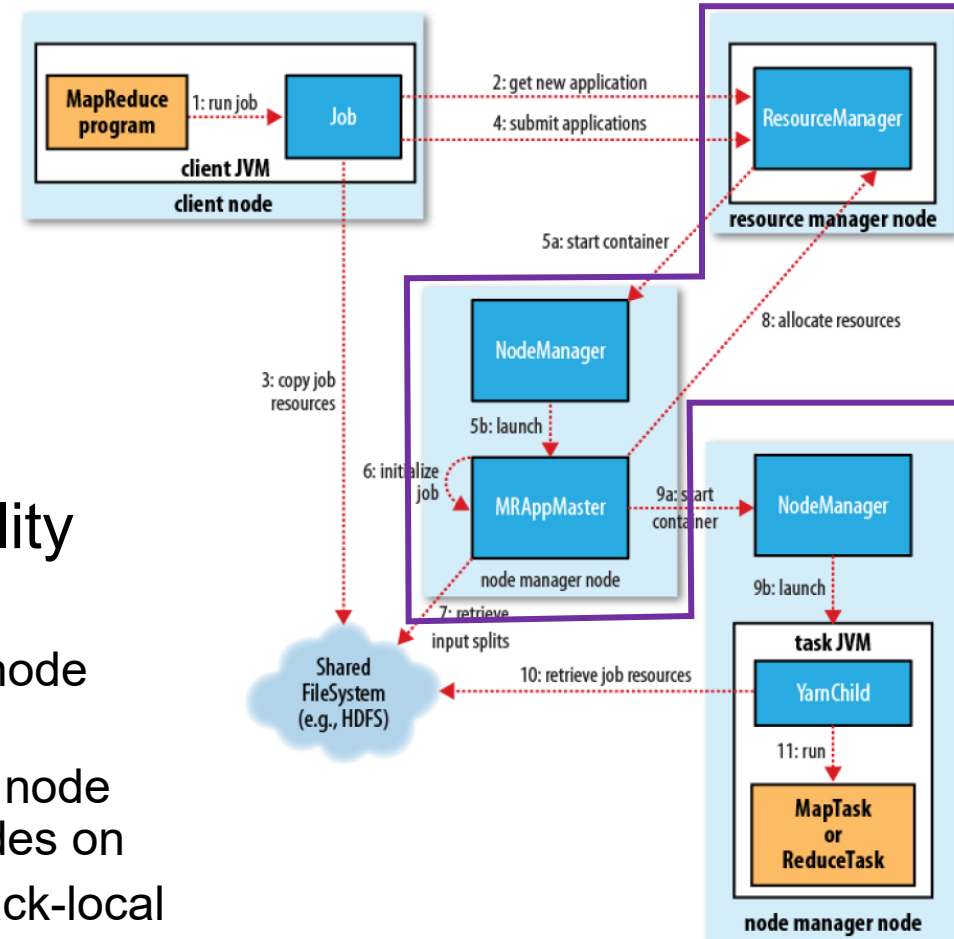
MapReduce execution

Task assignment

- 8: The AMP requests containers for map (and reduce) tasks
 - Requests for map tasks are made first and with higher priority
 - Reduce tasks are not made until 5% of the map tasks have completed

The RM tries to honor the data locality constraint for map tasks

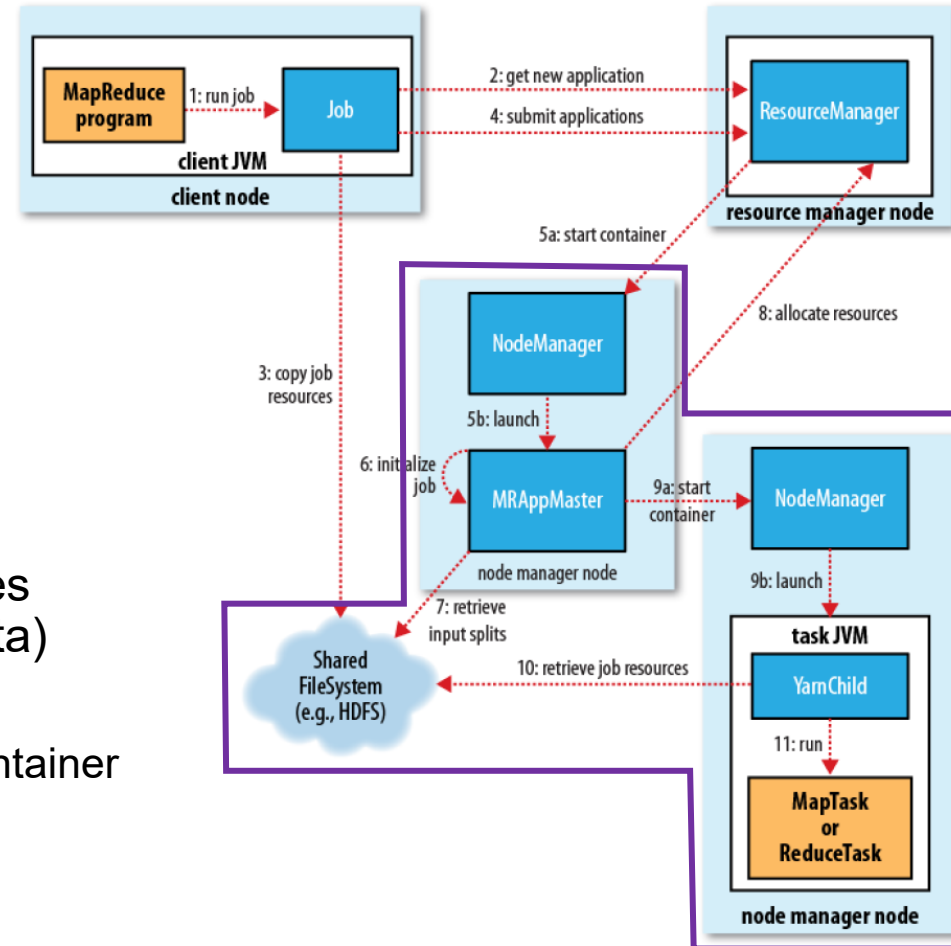
- *Data-local*: the task is run on the same node that the input split resides on
- *Rack-local*: the task is run on a different node in the same rack that the input split resides on
- Some tasks are neither data-local nor rack-local



MapReduce execution

Task execution

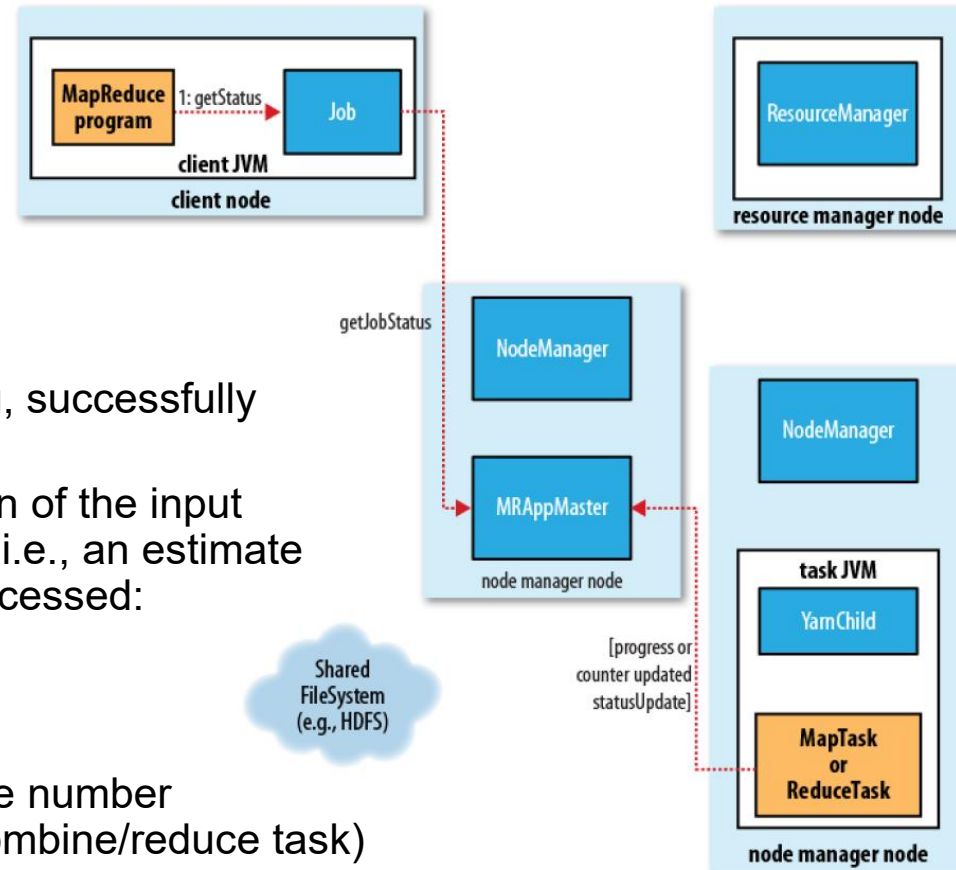
- 9a: The AMP starts the containers by contacting the NMs (in accordance to the assignments made by the RM)
- 9b: The container is initialized to run the map (or reduce) task
- 10: The task loads the required resources (i.e., configuration file, JAR file, input data)
- 11: The map (or reduce) task is run
 - Code execution is isolated within the container
 - Any bug/failure will not affect the NM



MapReduce execution

Progress and Status Updates

- Jobs may take from tens of seconds to hours to run
- A job and each of its tasks have a status, which includes:
 - the state of the job or task (e.g., running, successfully completed, failed)
 - the progress of maps (i.e., the proportion of the input that has been processed) and reduces (i.e., an estimate of the proportion of the reduce input processed:
 - 0-33% is shuffling;
 - 34-66% is sorting;
 - 67-100% is reducing)
 - the values of the job's counters (e.g., the number of records in input/output for the map/combine/reduce task)
 - a status message or description



MapReduce execution

Job Completion

- The user is notified of the job completion
- Job statistics and counters are printed
- The AMP and the task containers clean up their working state
 - E.g., intermediate output is deleted
- Job information is archived by the job history server

Coping With Node Failures

Failure of a worker (detected and managed by the AMP)

- The tasks assigned to this worker will have to be redone
 - In case of Reduce failure, the Map tasks that produced its input may have to be redone
- The AMP:
 - Sets the status of each failed tasks to idle
 - Reschedules them on a Worker when one becomes available

Failure of the AMP (detected and managed by the RM)

- Job history is used to recover the state of the tasks that were already run
- Worst case: the entire map-reduce job must be restarted

MapReduce "Runtime"

Handles scheduling

- Assigns workers to map and reduce tasks

Handles "data distribution"

- Gets data to the workers

Handles synchronization

- Gathers, sorts, and shuffles intermediate data

Handles errors and faults

- Detects container failures and restarts

Everything happens on top of a DFS

Evolving MapReduce

How did we get from Hadoop's MapReduce to Spark?

Limitations of Hadoop's MapReduce

Hadoop's MapReduce is the result of many years of development

- Hadoop 1.0 released in 2012
- Adopted by Yahoo in 2008
- Based on a paper by Google from 2004
- Who started working on MapReduce in 2000

In the meanwhile, a lot changed

- Hardware
 - RAM became cheaper and more available; multi-core machine became standard
- Software
 - HDFS and YARN enabled professionals to put their effort on the development of analytical tools
- Data and user needs
 - Big Data became everyone's problem and velocity became a hard requirement

Limitations of Hadoop's MapReduce

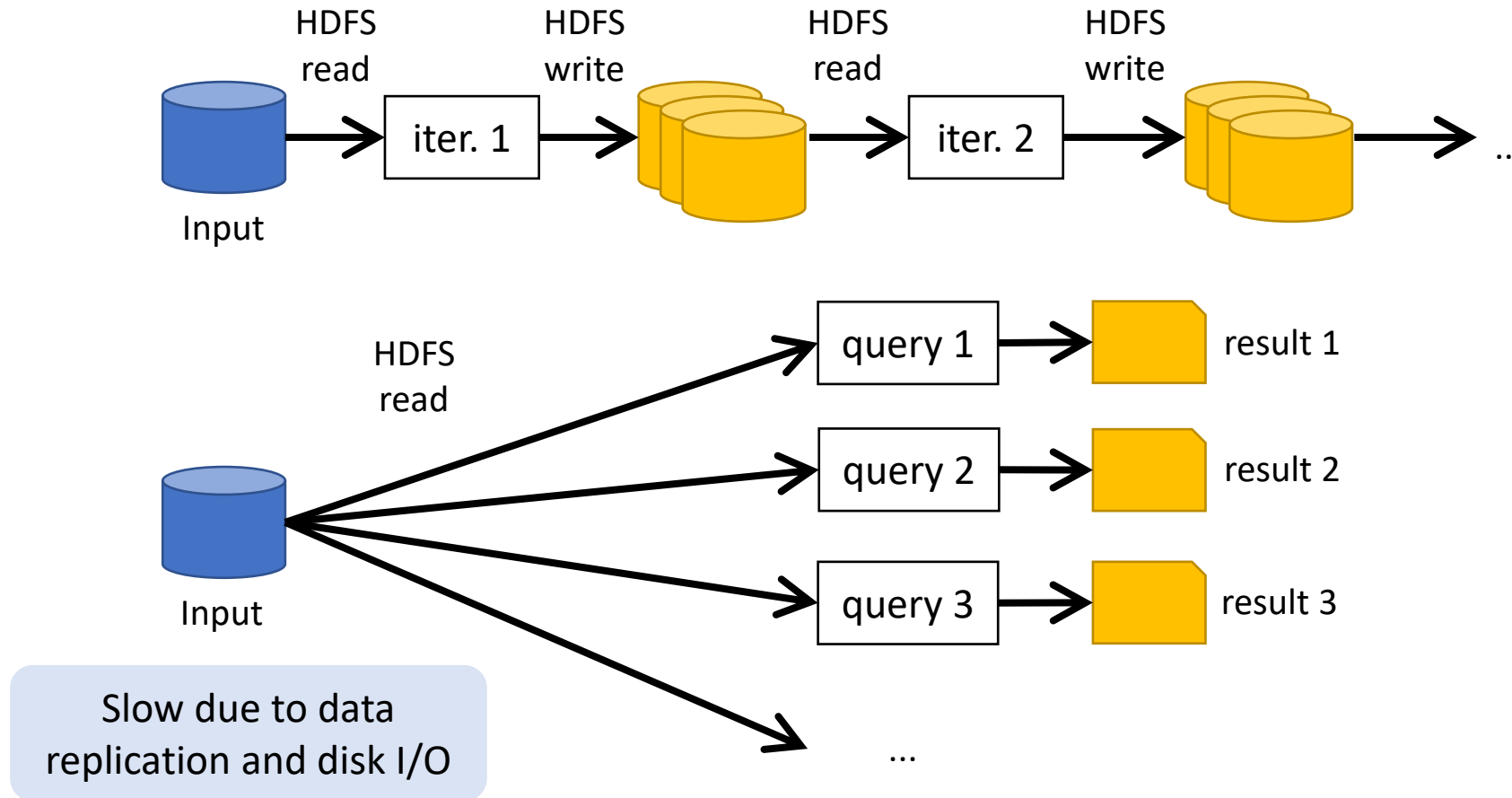
Hardware resources are not adequately exploited

- Every MapReduce job ends by saving results to HDFS, which is slow
- No possibility to keep data in RAM
- Multi-core architecture not easily exploited
- End-result: it's slow!

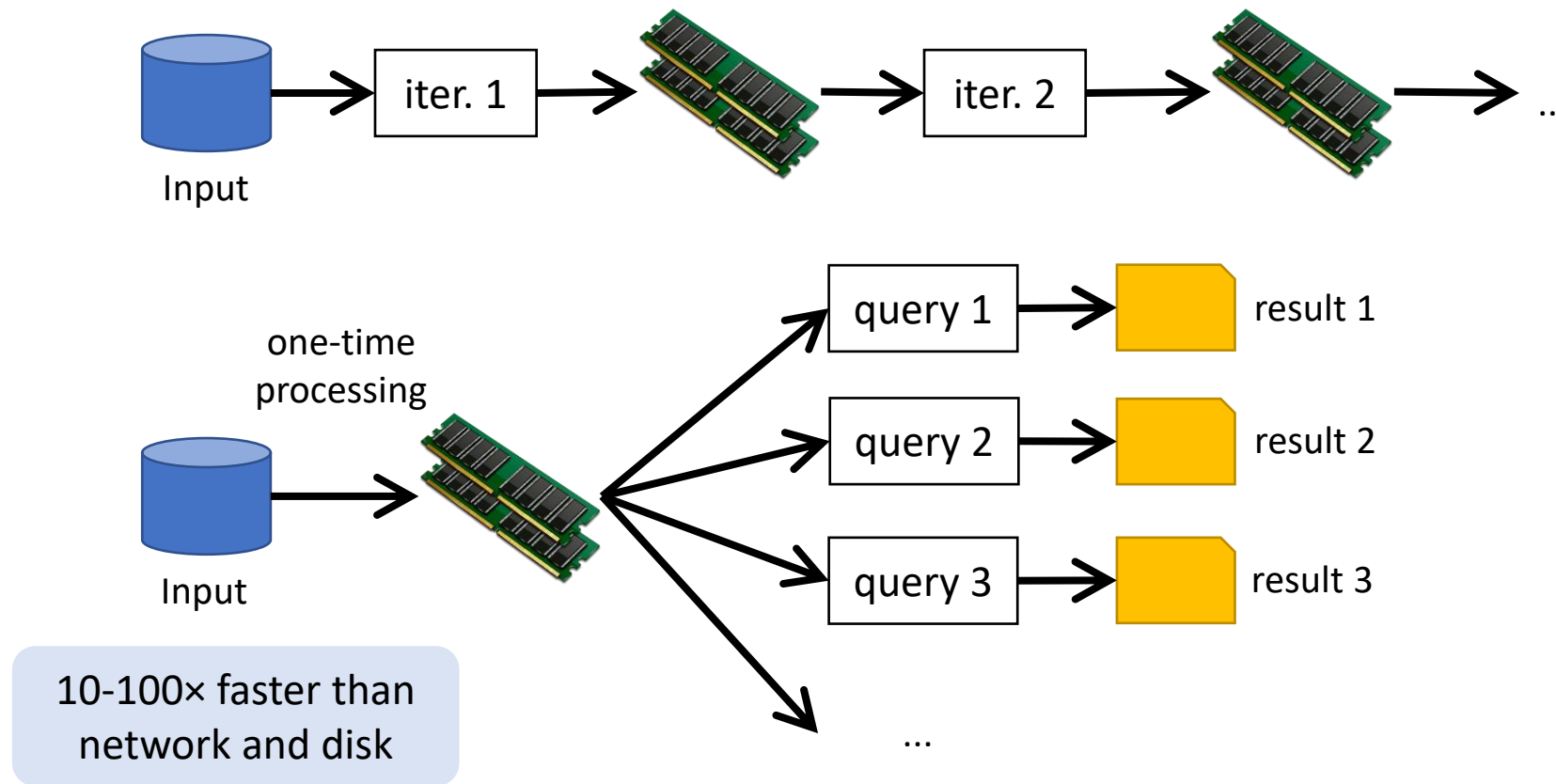
Strict paradigm

- Everything has to fit into Map and Reduce → programming isn't straightforward
- Complex algorithms will take multiple jobs and passes on hard disk
- Not suitable for iterative algorithms or interactive processing

Data processing in MapReduce



Data processing in Spark



Enter Apache Spark

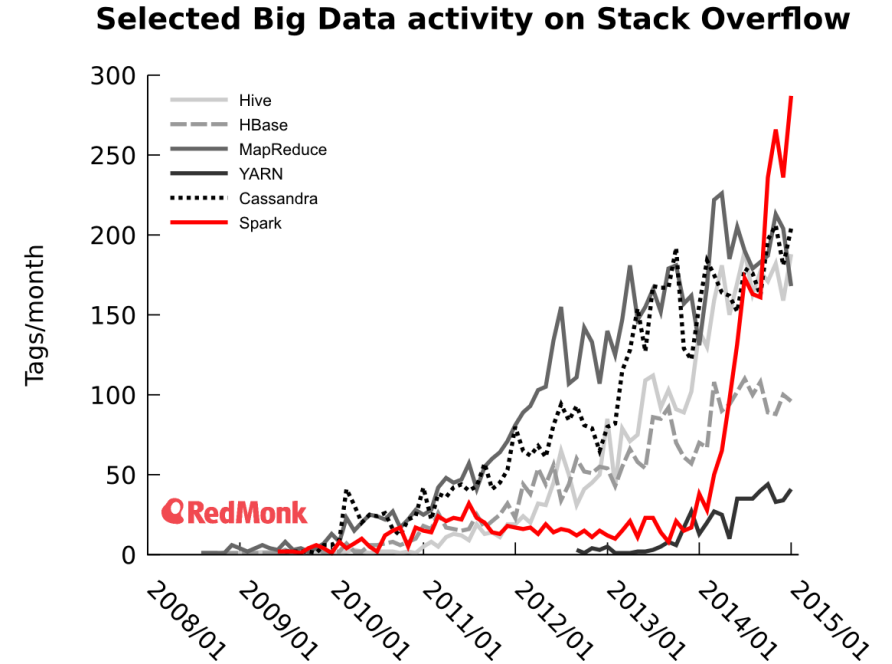
Spark history

Spark project started in 2009

- Developed at UC Berkeley's AMPLab by Matei Zaharia
- Originally built to test how Apache Mesos works
- Focused on:
 - Iterative programs (Machine Learning)
 - Interactive querying
 - Unifying real time and batch processing

Open sourced 2010, Apache project from 2013

- In 2014, Zaharia founded Databricks
- Current version: 3.3
- Written in Scala; supports Java, Python, .NET, R



Spark's innovations

A data structure called RDD (Resilient Distributed Dataset) is introduced

- Essentially, a set of data items (of any format) that is automatically distributed
- All operations are defined on RDDs
- No more separation between Map and Reduce

A rich set of APIs is provided to easily develop pipelines

- Depending on the type of operations, the system automatically defines the (map-style or reduce-style) tasks
- The execution plan of a job takes the form of a DAG

Better usage of RAM

- Results are saved to disk only when the processing has finished
- RDDs can be temporarily cached

Finer control over resource assignment (cores and memory)

RDD creation

RDDs can be created in two ways

- By [loading an external dataset](#)
 - Many sources (HDFS, S3, local file system, HBase, Cassandra, Hive, etc.)
 - Many formats (text files, SequenceFiles, Avro, etc.)
- By [distributing a collection of objects](#) (e.g., a list or set)

Example (in Python)

- `sc = new SparkContext`
- `rddLines = sc.textFile("myFile.txt")`
- `rddLines2 = rddLines.filter(lambda line: "some text" in line)`

RDD operations

RDDs offer two types of operations: *transformations* and *actions*

Transformations construct a new RDD from a previous one

- E.g.: map, flatMap, reduceByKey, filtering (see previous slide), etc.
- <https://spark.apache.org/docs/latest/programming-guide.html#transformations>

Actions compute a result that is either returned to the driver program or saved to an external storage system (e.g., HDFS)

- E.g.: saveAsTextFile, count, collect, etc.
- <https://spark.apache.org/docs/latest/programming-guide.html#actions>

RDD operations

Each operation expresses a dependency $A \rightarrow B$ from an RDD A (input) to an RDD B (output)

Two kinds of dependencies:

- **Narrow dep.**: the data from partition a_i of A ends up in **one** partition b_j of B
 - E.g.: map, filter
- **Wide dep.**: the data from partition a_i of A ends up in **many (possibly every)** partition of B
 - E.g.: groupByKey, reduceByKey

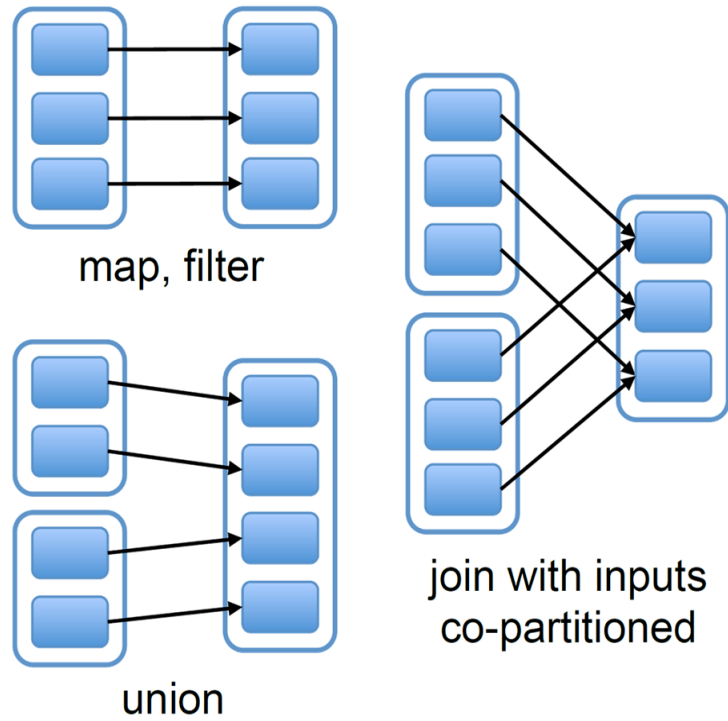
Narrow dependencies allow for **pipelining** on one cluster node (more optimized execution)

Wide dependencies can't be pipelined, as they require **data shuffling**

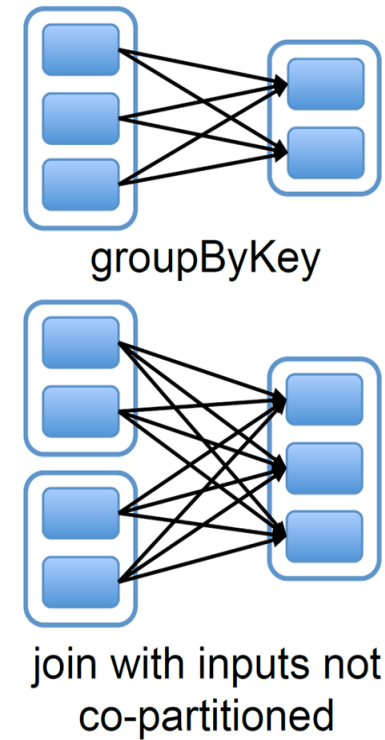
In case of failure, narrow dependencies require less partitions to be recomputed

RDD dependencies

Narrow Dependencies



Wide Dependencies



DAG

Based on the user application and on the lineage graphs, Spark computes a **logical execution plan** in the form of a DAG

- Which is later transformed into a physical execution plan

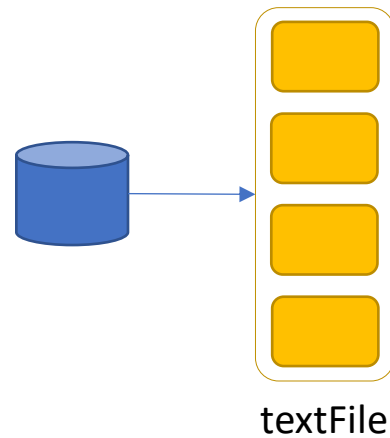
The DAG (Directed Acyclic Graph) is **a sequence of computations performed on data**

- Nodes are **RDDs**
- Edges are operations on RDDs
- The graph is Directed: transformations from an RDD A to an RDD B
- The graph is Acyclic: transformations cannot return an old RDD

DAG example

Word count in Scala

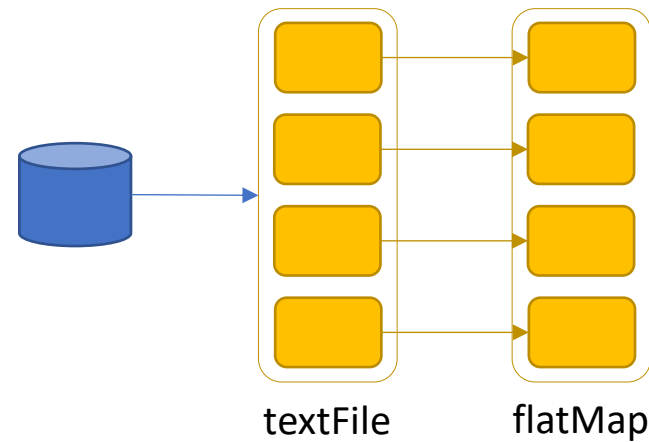
- `textFile = sc.textFile("hdfs://...")`



DAG example

Word count in Scala

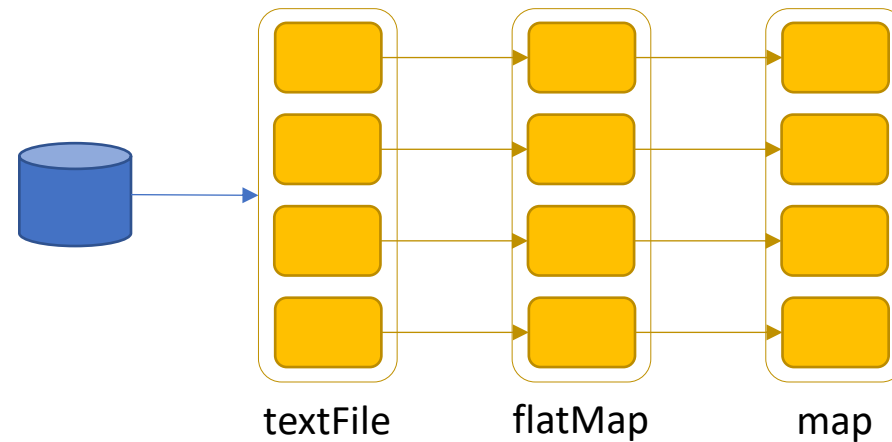
- `textFile = sc.textFile("hdfs://...")`
- `counts = textFile.flatMap(line => line.split(" "))`



DAG example

Word count in Scala

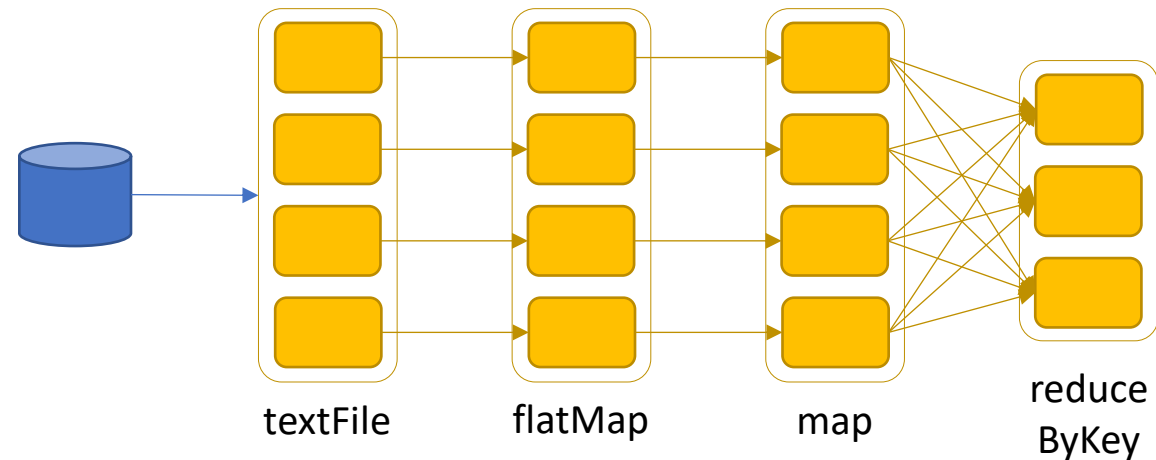
- `textFile = sc.textFile("hdfs://...")`
- `counts = textFile.flatMap(line => line.split(" "))
 .map(lambda word: (word, 1))`



DAG example

Word count in Scala

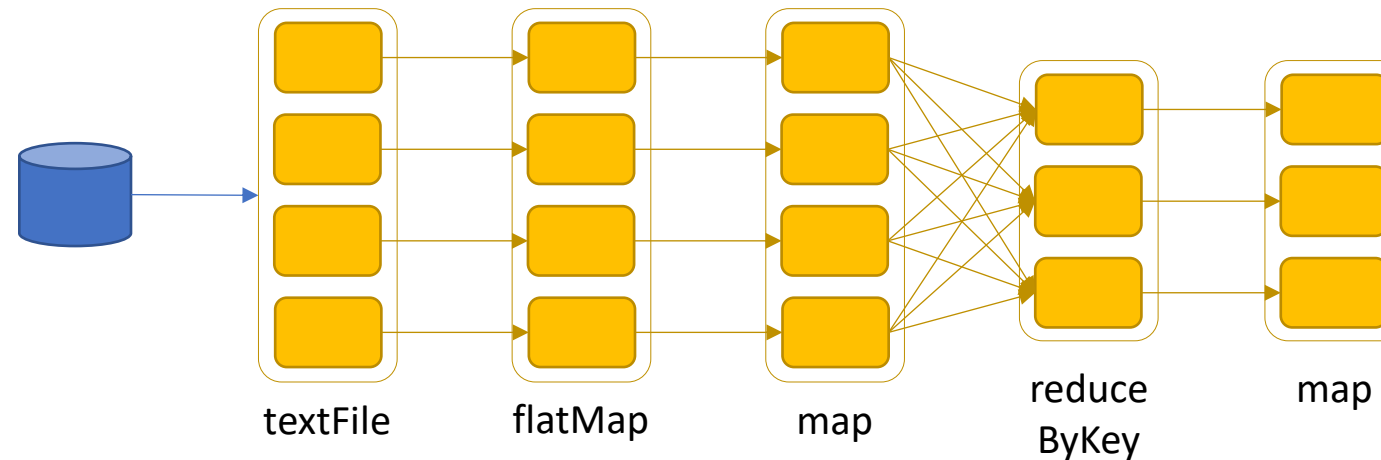
- `textFile = sc.textFile("hdfs://...")`
- `counts = textFile.flatMap(line => line.split(" "))`
 `.map(lambda word: (word, 1))`
 `.reduceByKey(lambda a,b: a + b)`



DAG example

Word count in Scala

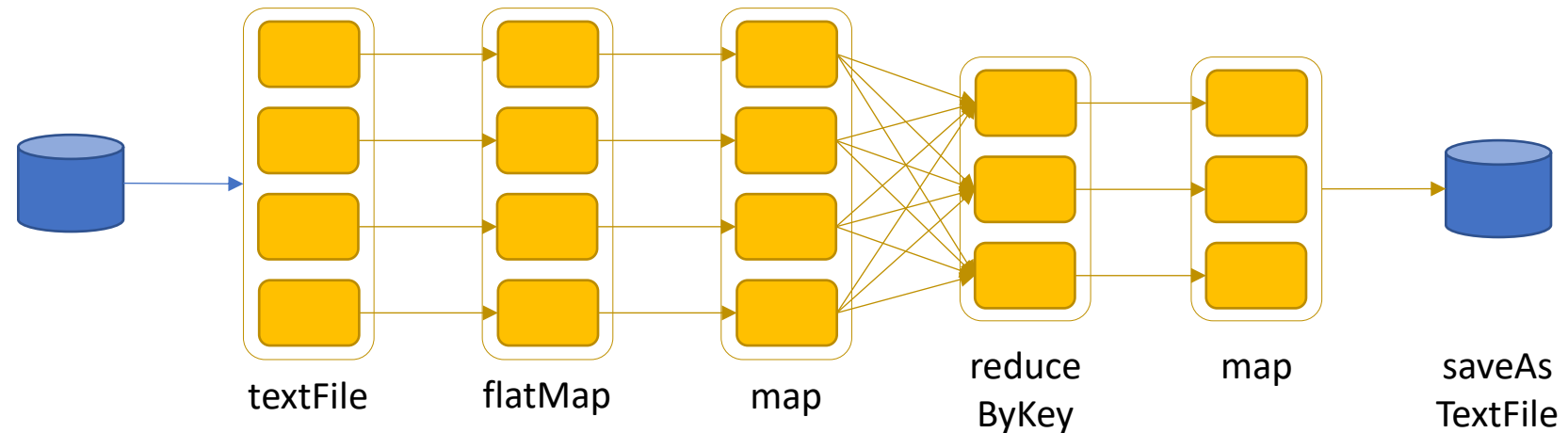
- `textFile = sc.textFile("hdfs://...")`
- `counts = textFile.flatMap(line => line.split(" "))`
 `.map(lambda word: (word, 1))`
 `.reduceByKey(lambda a,b: a + b)`
 `.map(lambda k,v: k.upper(), v)`



DAG example

Word count in Scala

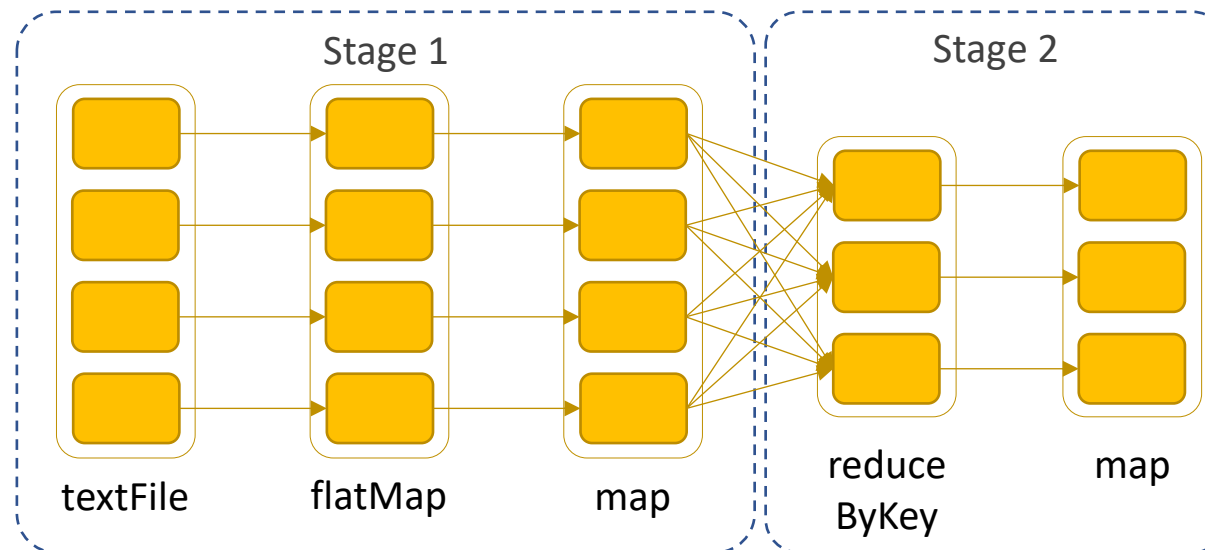
- `textFile = sc.textFile("hdfs://...")`
- `counts = textFile.flatMap(line => line.split(" "))`
 `.map(lambda word: (word, 1))`
 `.reduceByKey(lambda a,b: a + b)`
 `.map(lambda k,v: k.upper(), v)`
- **`counts.saveAsTextFile("hdfs://...")`**



DAG stages

The execution plan is compiled into physical **stages**

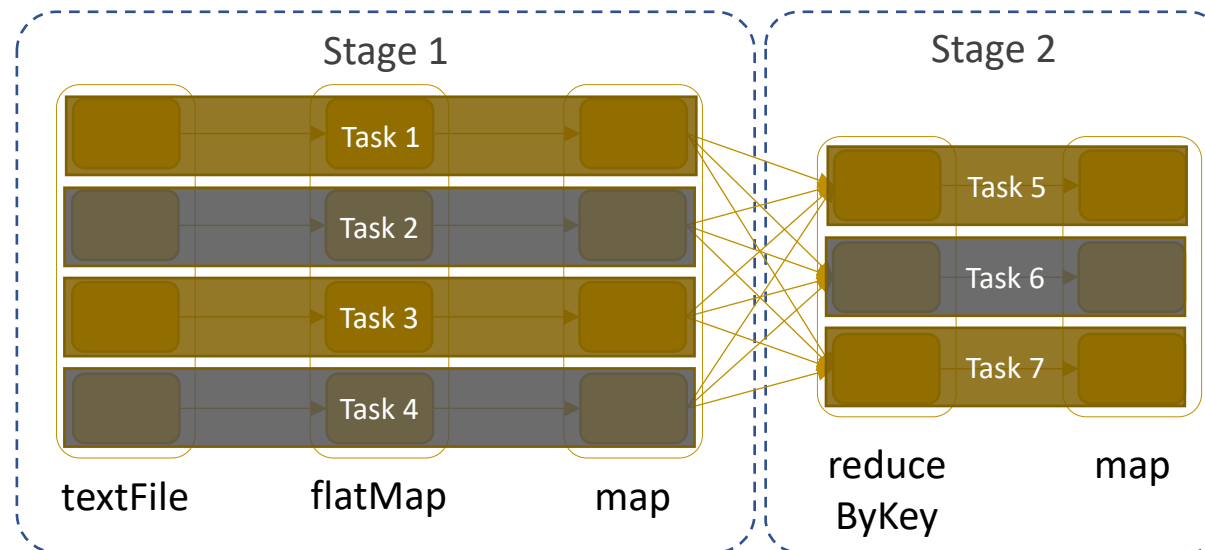
- Stages' boundaries are defined by shuffle operations
- Operations with narrow dependencies are pipelined as much as possible



DAG tasks

The fundamental unit of execution is the one of **tasks**

- A task is created for each partition in the new RDD
- Tasks are scheduled and assigned to the worker nodes based on data locality
- The scheduler can run the same task on multiple nodes in case of stragglers (i.e., slow nodes)



Application decomposition

Application

- Single instance of SparkContext that stores data processing logic and schedules series of jobs, sequentially or in parallel

Job

- Complete set of transformations on RDD that finishes with action or data saving, triggered by the driver application

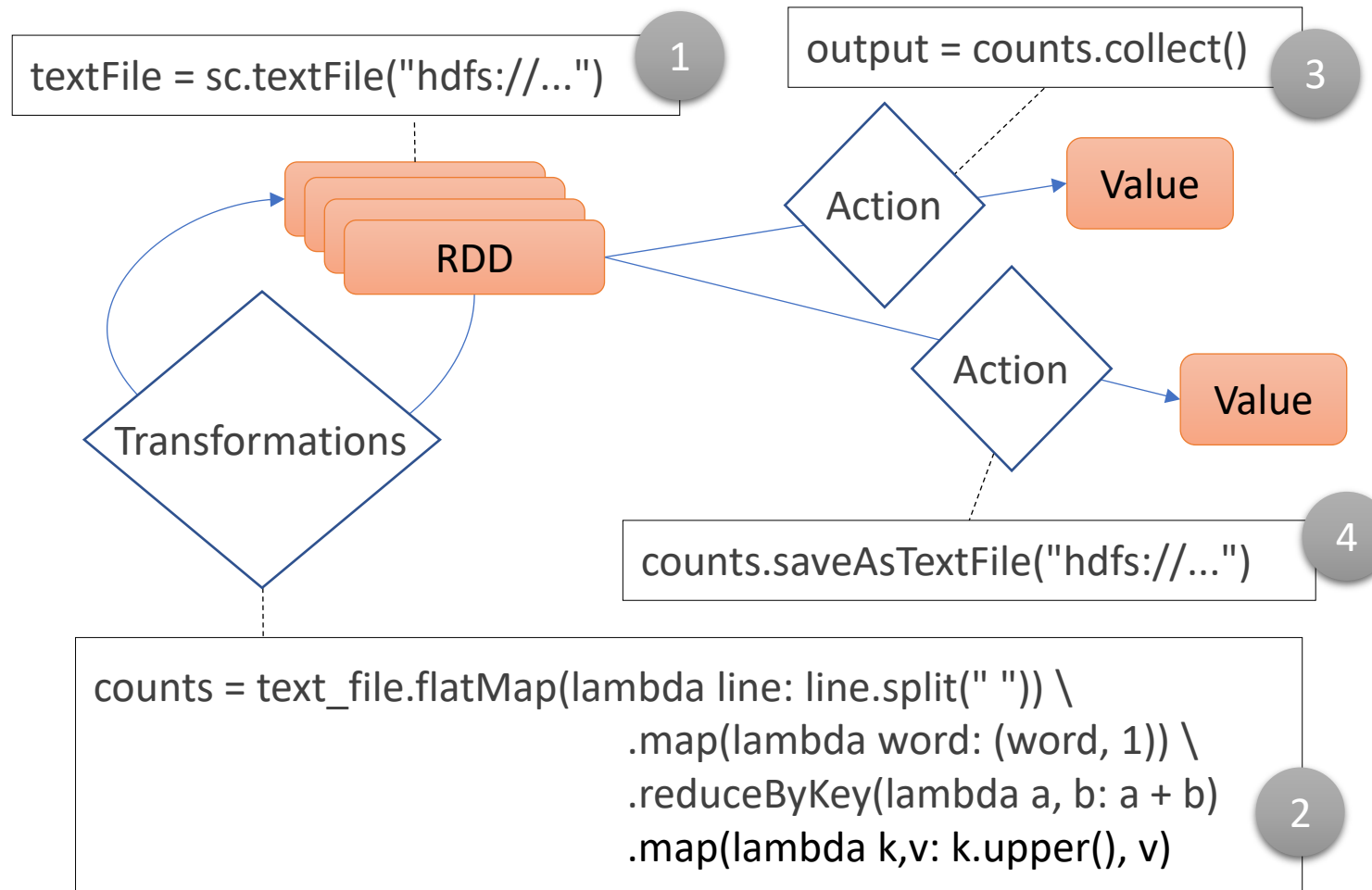
Stage

- Set of transformations that can be pipelined and executed by a single independent worker

Task

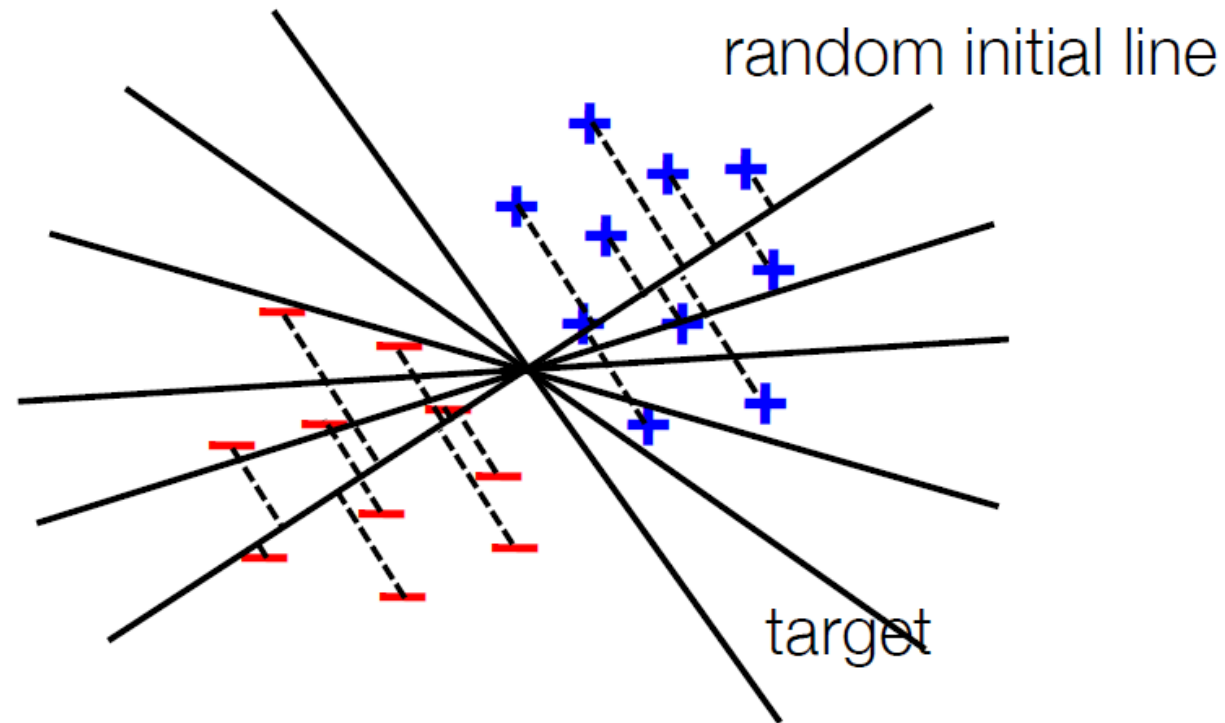
- Basic unit of scheduling: executes the stage on a single data partition

Conceptual representation



Example: logistic regression

Goal: find line separating two sets of points



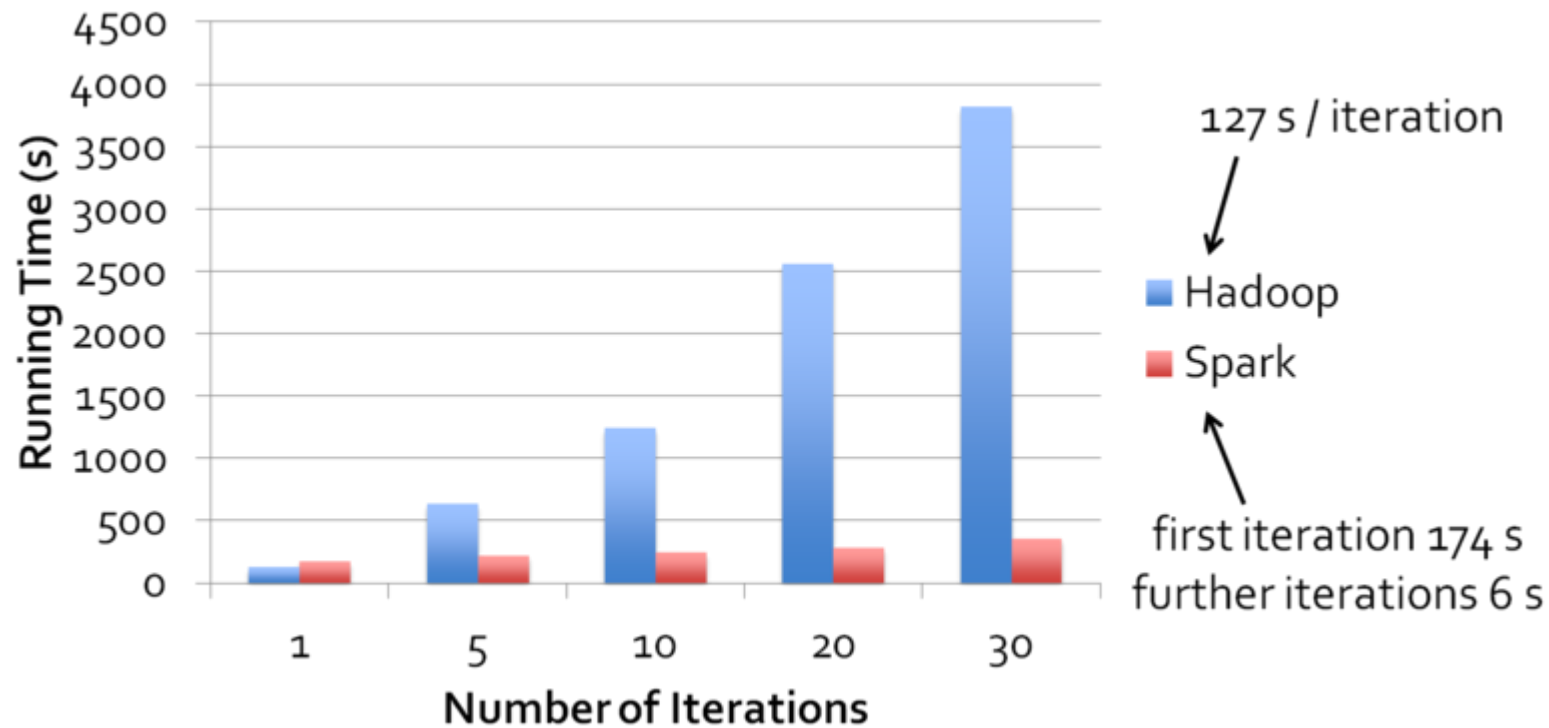
Example: logistic regression

Goal: find line separating two sets of points

- `val data = spark.textFile(...).map(readPoint).cache()`
- `var w = Vector.random(D)`
- `for (i <- 1 to ITERATIONS) {`
 - `val gradient = data`
 - `.map(p => (1 / (1 + exp(-p.y * (w dot p.x))) - 1) * p.y * p.x)`
 - `.reduce(_ + _)`
 - `w -= gradient`
- `println("Final w: " + w)`

Example: logistic regression

Goal: find line separating two sets of points



SQL-on-Hadoop

Why SQL?

SQL-on-Hadoop is a class of application tools that combine established SQL-style querying with newer Hadoop data framework elements

- MapReduce-style programming is the native and most comprehensive way to design jobs
- But SQL is a simple and widely understood language

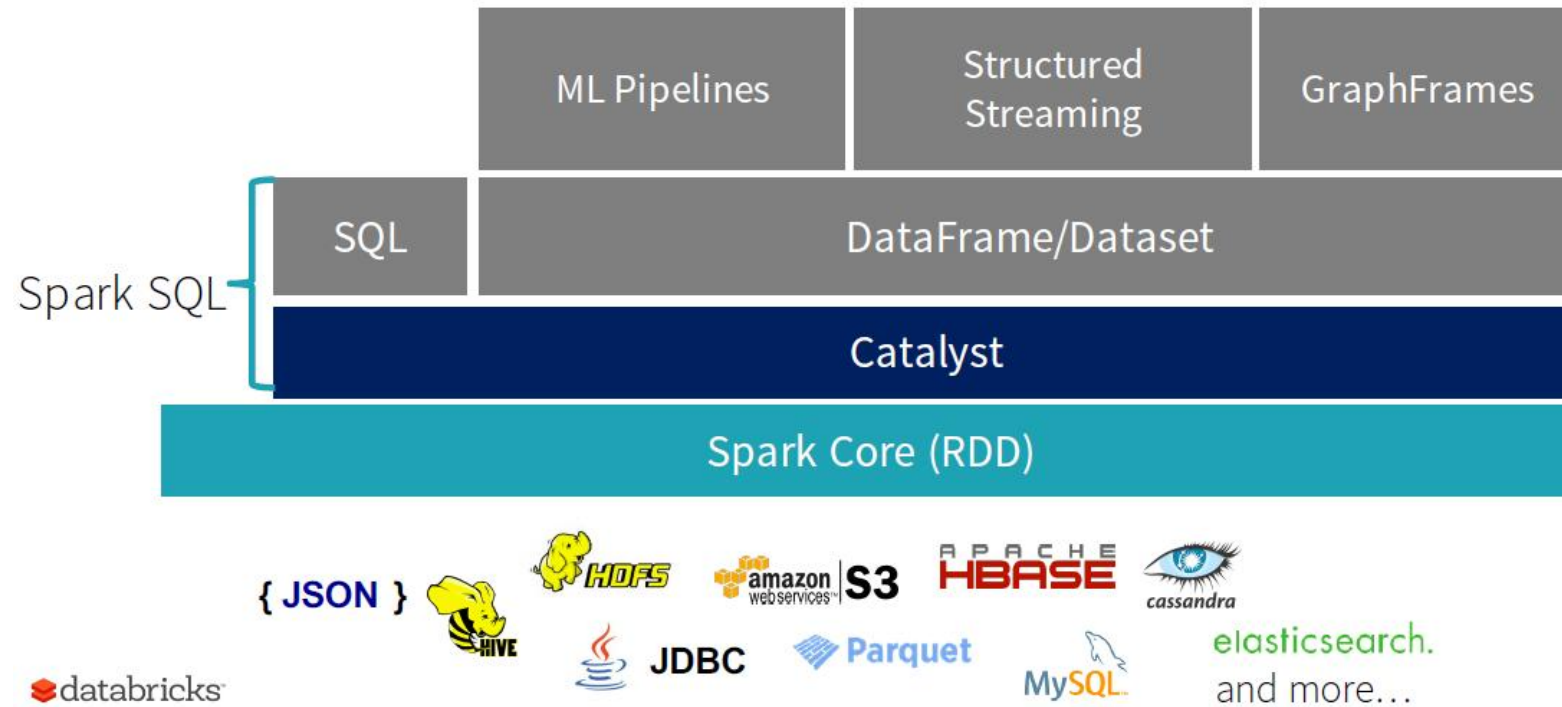
Con

- **SQL syntax imposes some limits**
 - Whereas RDDs enable any computation through user defined functions (UDFs)

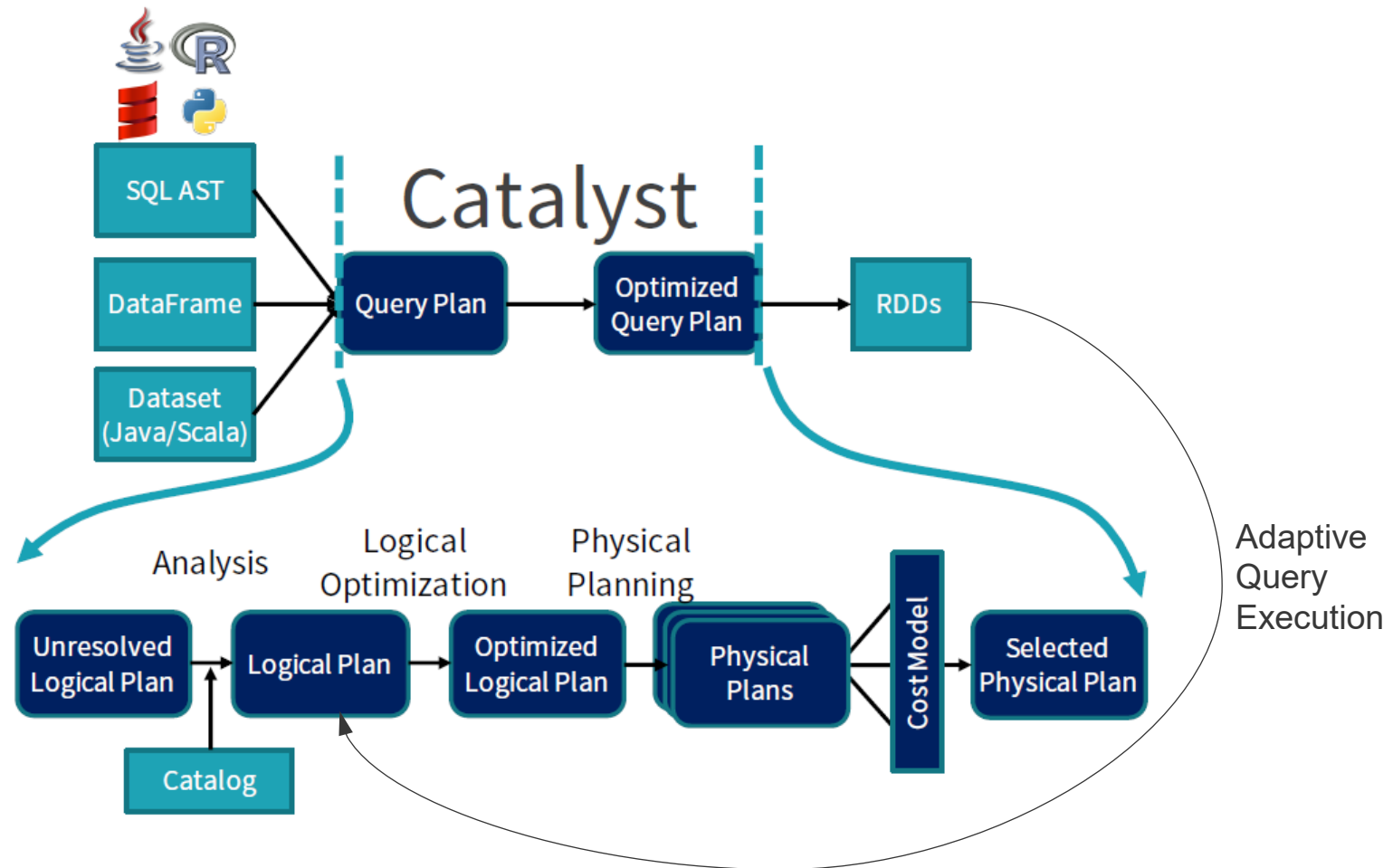
Pros

- The most common computations are supported
- Language simplicity, syntax broadly known
- **Opens the room to optimizations**
 - Hard to optimize a user defined function

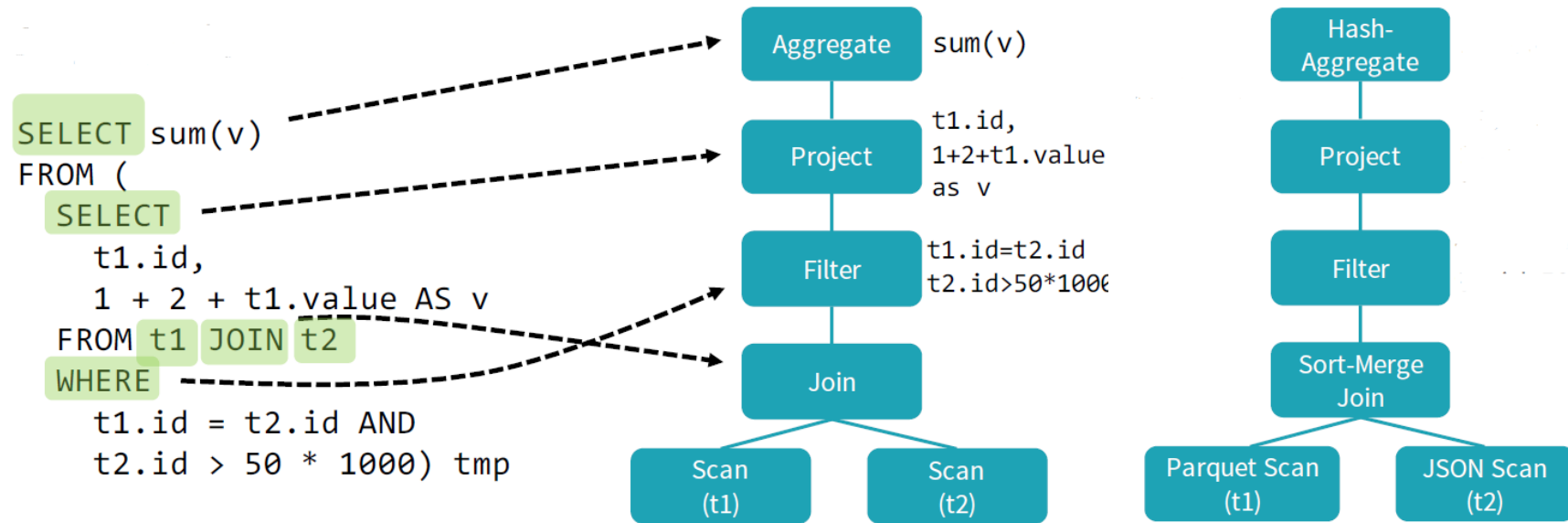
Spark structured



Catalyst



Logical and Physical Plan



Logical Plan

Describes **which** computations must be done

Physical Plan

Describes **which** computations must be done and **how** to conduct them (i.e., which algorithms are used)

Logical optimization

Based on rules

- **A rule is a function** that can be applied on a portion of the logical plan

Implemented as Scala functions

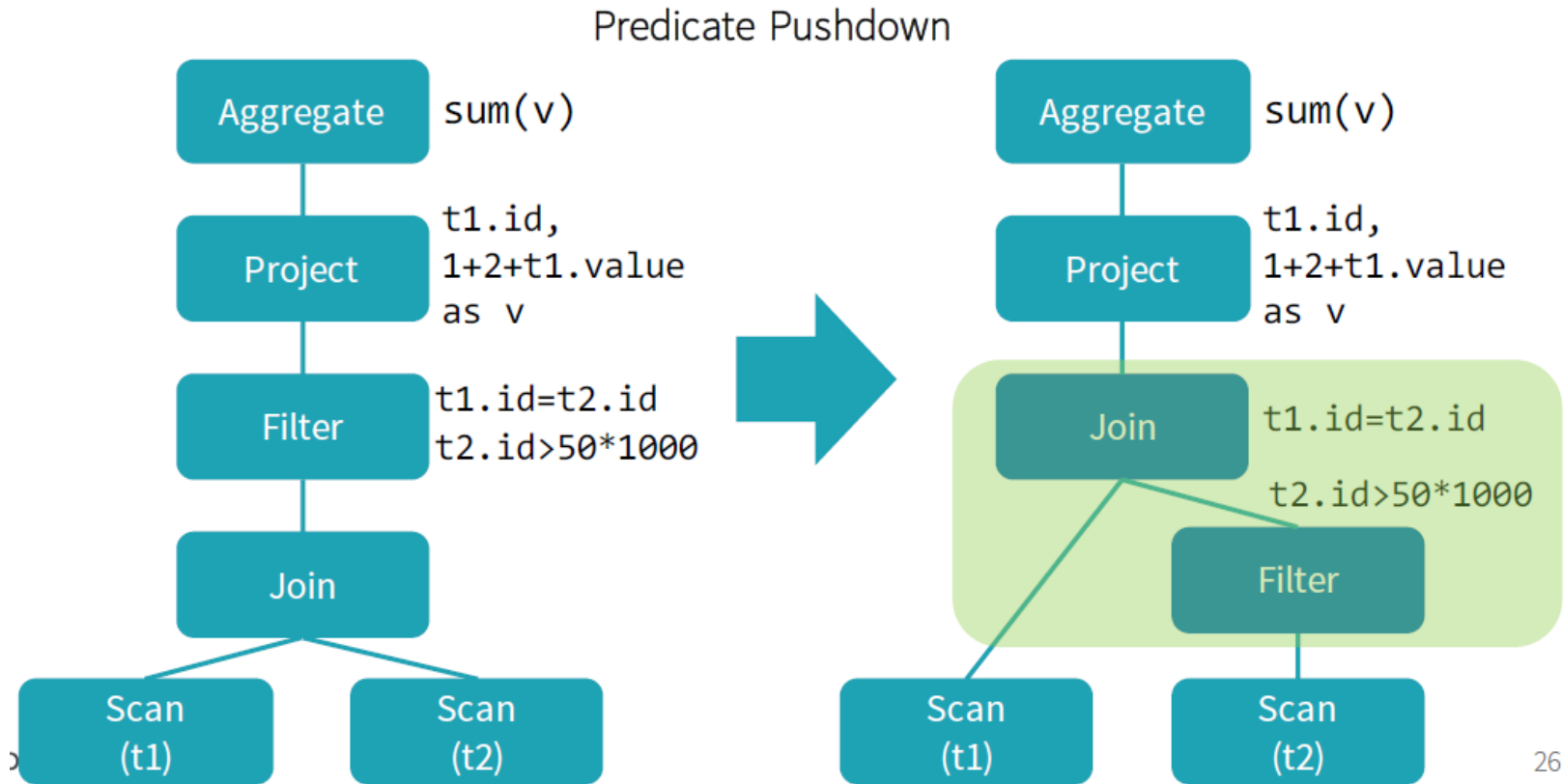
```
val expression: Expression = ...
expression.transform {
  case Add(Literal(x, IntegerType), Literal(y, IntegerType)) =>
    Literal(x + y)
}
```

Several types of rules

- **Constant folding**: resolve constant expressions at compile time
- **Predicate pushdown**: push selection predicates close to the sources
- **Column pruning**: project only the required column
- **Join reordering**: change the order of join operations

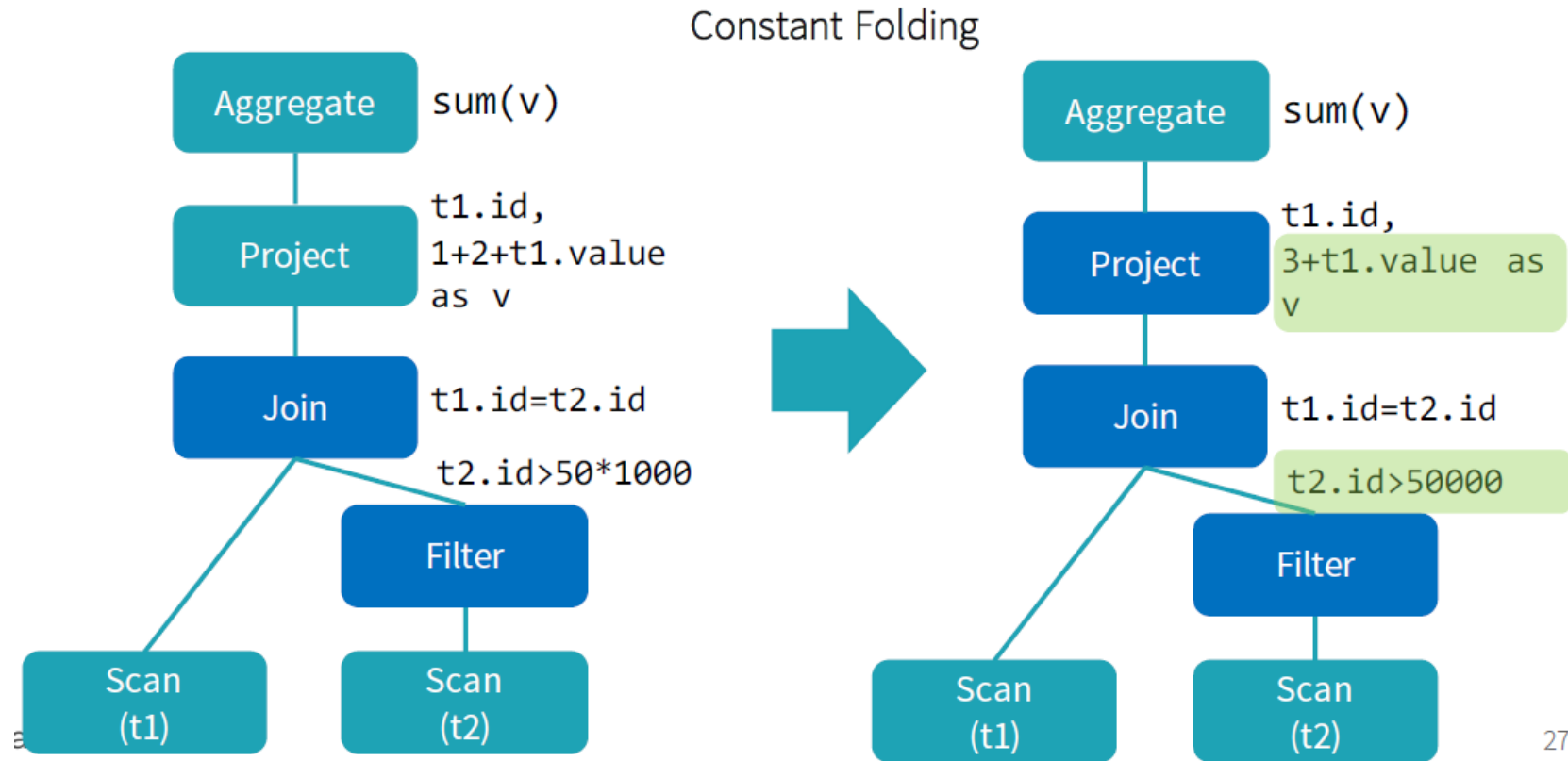
Applied recursively and iteratively until the plan reaches a *fixed point*

Logical optimization



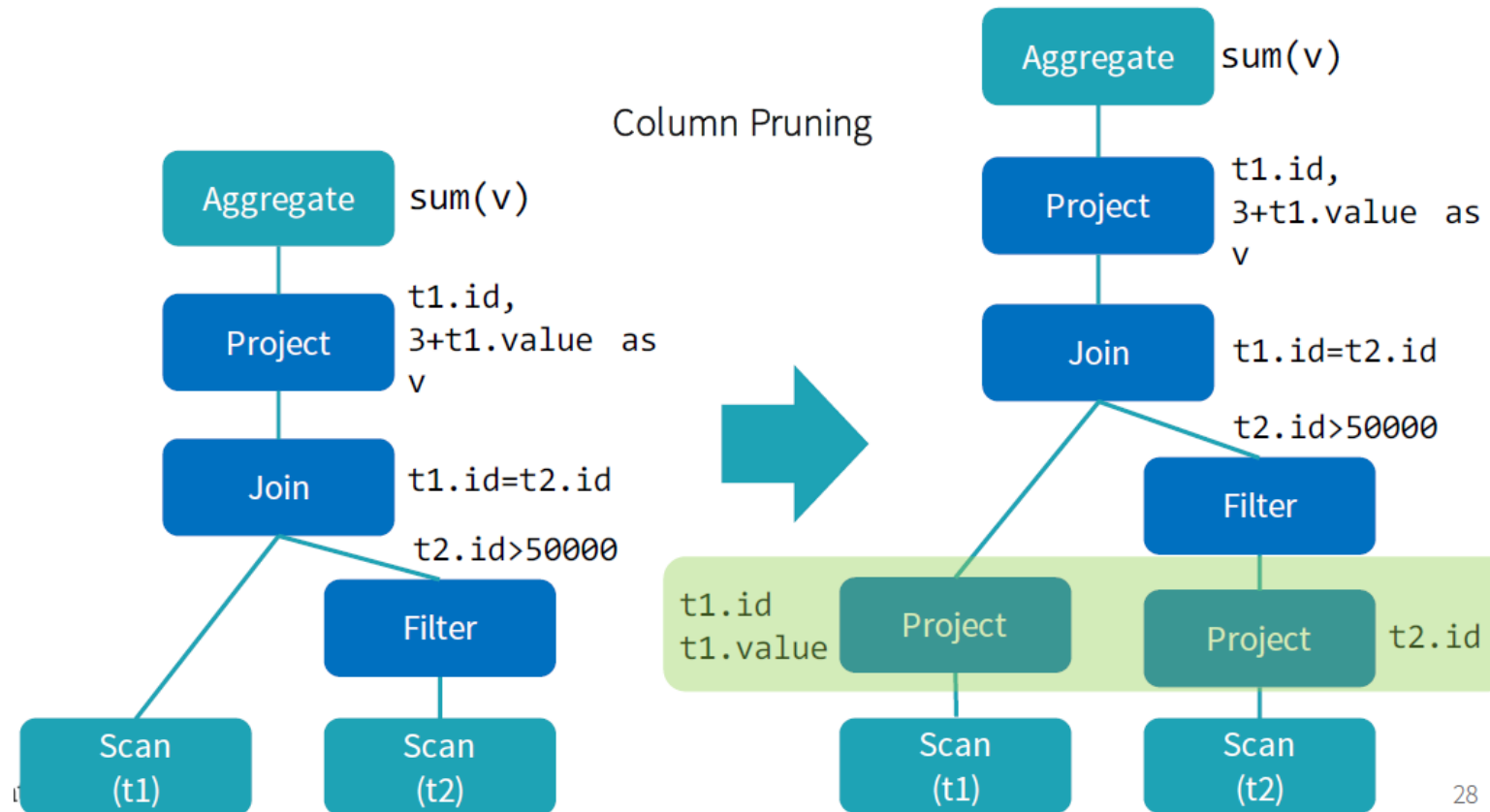
26

Logical optimization



27

Logical optimization



28

Physical optimization

One or more physical execution plans are defined

- A cost model is used to select the best one
- $\text{Cost}_{\text{table}} = \alpha \cdot \text{cost}_{\text{CPU}} + (1-\alpha) \cdot \text{cost}_{\text{IO}}$
 $= \alpha \cdot \text{cardinality} + (1-\alpha) \cdot \text{size}$

Rule based physical optimizations

- Operation pipelining (similarly to RDDs)
- Predicate push-down (e.g., filtering performed during physical scan)

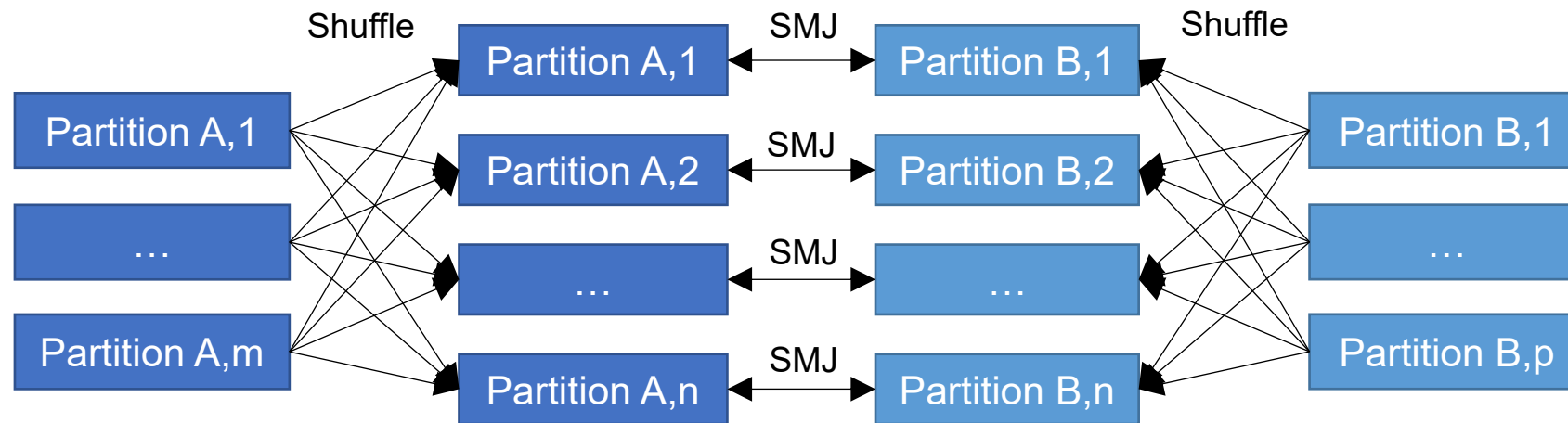
Join method selection

- Next slide

Physical optimization - Join methods

Shuffle Sort Merge join

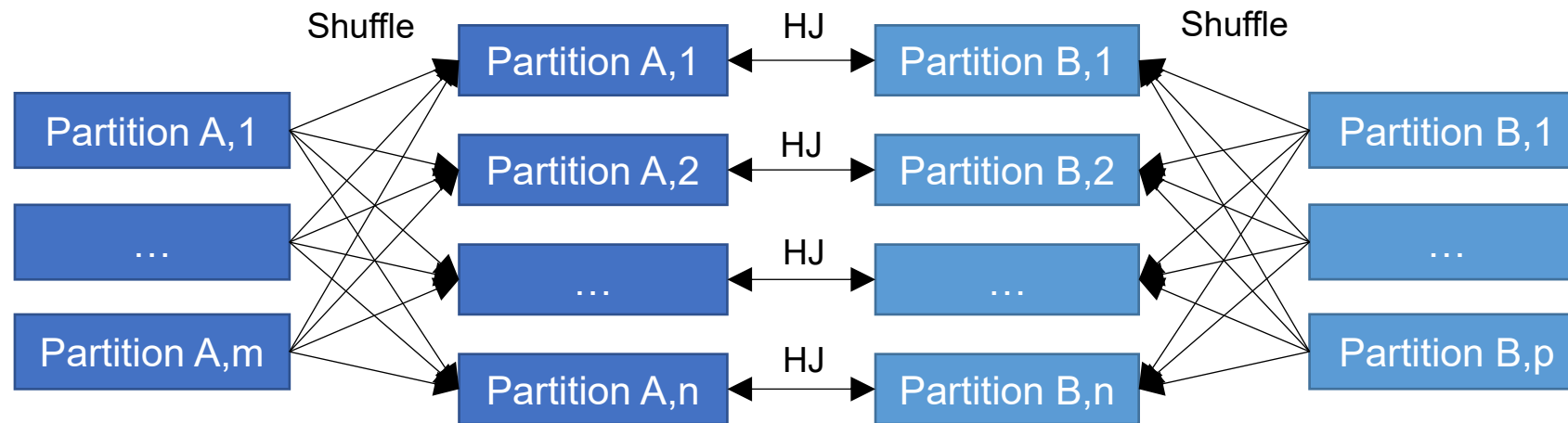
- Each table is repartitioned based on the join key
- Sort Merge join (SMJ) is applied
 - The keys on each partition are sorted
 - The join is carried out by iterating on the sorted lists



Physical optimization - Join methods

Shuffle Hash join

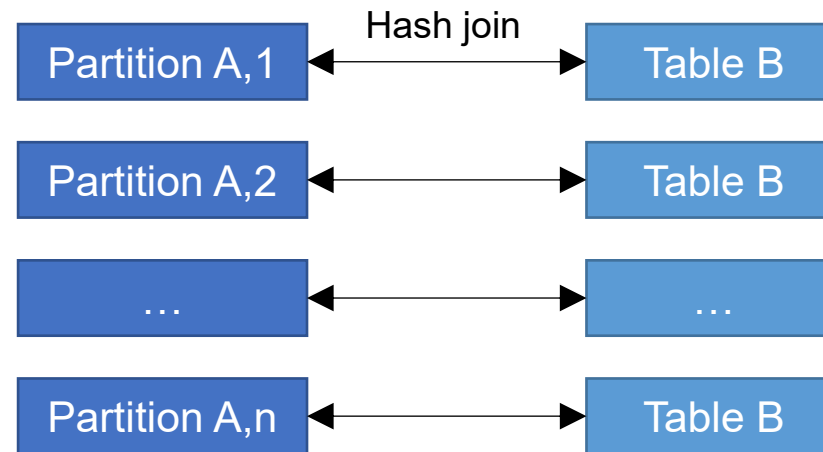
- Each table is repartitioned based on the join key
- Hash join (HJ) is applied
 - The Hash table of the smaller partition is loaded into memory
 - For each record of the bigger partition, the matching records are found by using the Hash table



Physical optimization - Join methods

Broadcast Hash join

- The smaller table is replicated for each partition and fully loaded in memory



Physical optimization - Join method selection

Conditions for **Broadcast Hash join** to be selected

- One DF is smaller than a certain threshold
 - `spark.sql.autoBroadcastJoinThreshold` (default: 10MB; some people increase it to GB levels)

Conditions for **Shuffle Hash join** to be selected

- Conditions for Broadcast Hash join do not apply
- One DF is 3 times smaller than the other
- The partitions of the smaller DF are small enough for their hash tables to fit into memory

Conditions for **Shuffle Sort Merge join** to be selected

- Conditions for Broadcast Hash join and Shuffle Hash join do not apply

Join methods are under continuous refinement

- A few more join methods exist for certain situations (e.g., cartesian product)

Physical optimization - What determines the cost?

Catalyst only considers the size and the cardinality of tables

Other important factors

- Network throughput
- Disk throughput
- Allocation of resources
 - Number of Executors
 - Number of Cores per Executor
- Allocation of tasks
 - Data locality probability

Research work: defining a more accurate, probabilistic cost model

- Baldacci, L., & Golfarelli, M. (2018). A cost model for Spark SQL. IEEE Transactions on Knowledge and Data Engineering, 31(5), 819-832.
- Gallinucci, E., & Golfarelli, M. (2019). SparkTune: tuning Spark SQL through query cost modeling. EDBT 2019: 546-549.

Adaptive Query Execution (AQE)

Introduced with version 3.0

Main idea

- The execution plan is not final
- Reviews are made at each stage boundary
- Additional optimizations are possibly applied, given the information available on the intermediate data

AQE can be defined as a layer on top of the Spark Catalyst which will modify the Spark plan on the fly

Drawbacks

- The execution stops at each stage boundary for Spark to review its plan
 - But the gain in performance is usually worth the time spent to revisit the plan
- The execution plan is more difficult to read
 - The original multi-stage job becomes a set of single-stage jobs

AQE - Adaptive Number of Shuffle Partitions

Spark SQL used to set a default number of 200 partitions at each stage.
AQE automatically adjusts it at runtime.

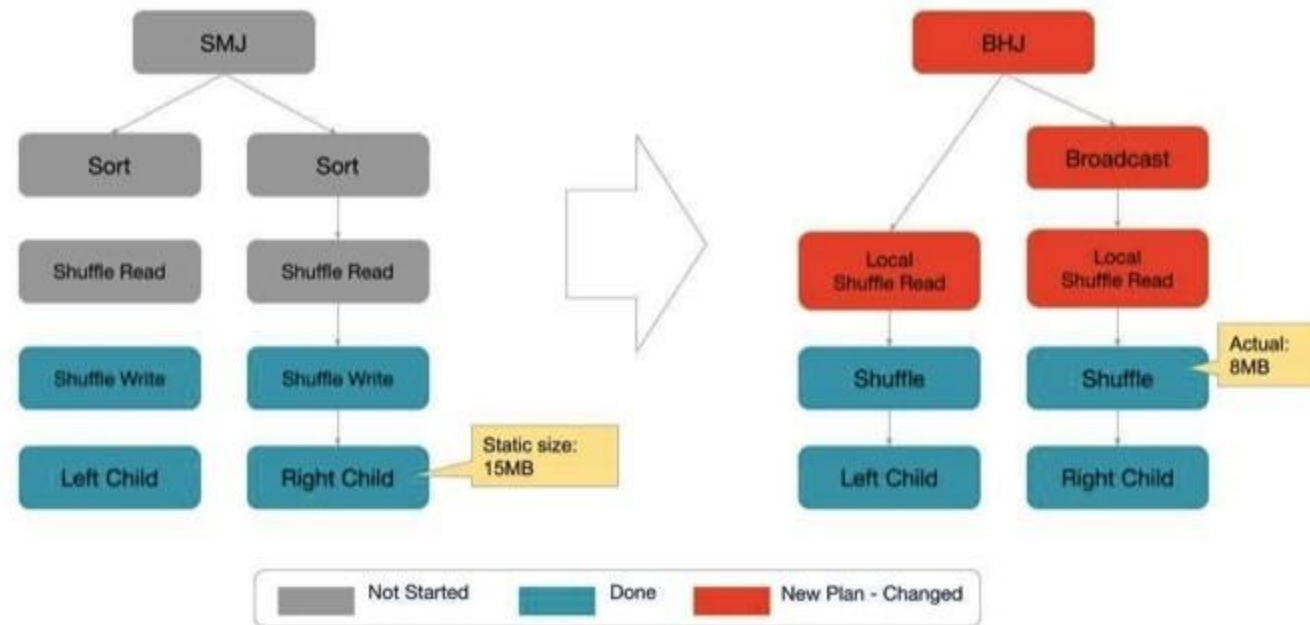
Tasks: Succeeded/Total
200/200
200/200
2/2
200/200
200/200
2/2



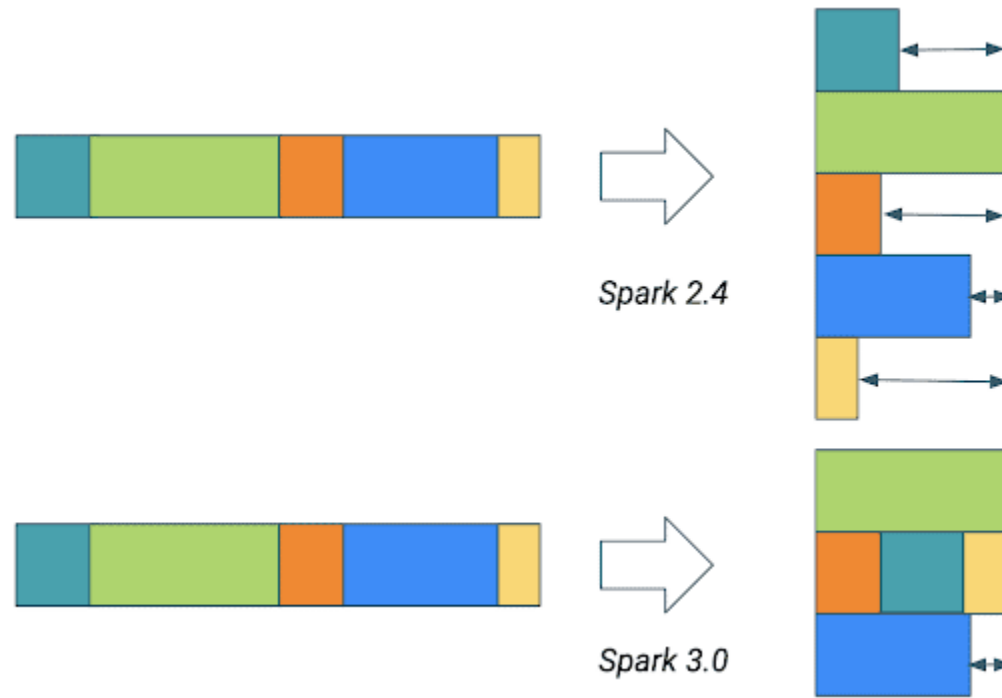
Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
1/1 (1 skipped)	1/1 (2 skipped)
1/1 (2 skipped)	1/1 (3 skipped)
1/1 (1 skipped)	1/1 (2 skipped)
1/1	2/2
1/1	2/2
1/1	2/2
1/1	2/2
1/1	2/2

AQE - Dynamically Converting Sort Merge Joins to Broadcast Joins

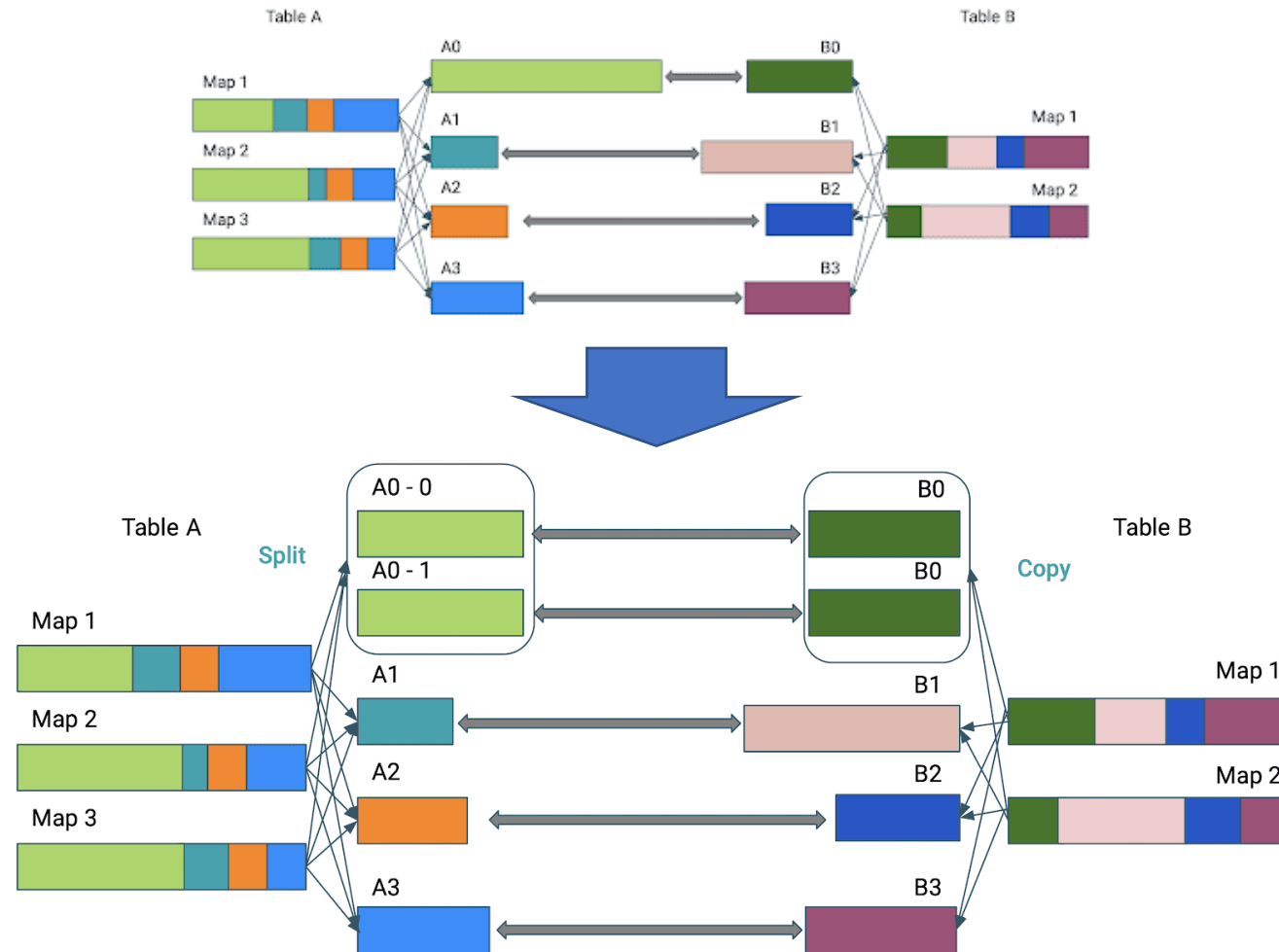
Dynamic switch of join strategies based on actual table sizes.



AQE - Dynamically Coalesce Shuffle Partitions



AQE - Dynamically Optimize Skewed Joins



Beyond Spark

Beyond Spark

Spark is the standard *de facto* for batch analysis on big data

- Jobs are executed on-demand: resources are used only when computations are requested
- The execution engine is internal: Spark is directly in charge of executing tasks
- Storage is separated: Spark only reads from/writes to storage systems

Alternative solutions do exist

- MPP-style (Massively Parallel Processing) tools
 - Resources are always allocated: a service runs on all (many) nodes, ready to run jobs/tasks
 - No startup time, but resources remain allocated even if not used
- External execution
 - User requests (SQL queries) are translated to jobs executable with other tools
- Included storage
 - The tool handles both execution and storage
 - Usually a (distributed) (NoSQL) DBMS

Category	Query tool	Data storage	Execution engine	Allocation of resources
Operational	NoSQL	Own	Own	Per service
Operational	HBase	DFS	Own	Per service
Interactive	Impala	DFS	Own (MPP)	Per service
Interactive	Drill	Any	Own (MPP)	Per service
Batch	Spark	Any	Own (YARN)	On demand
Batch	Hive	DFS	MapReduce, Tez, Spark	On demand